# TTL 4-Banger Calculator

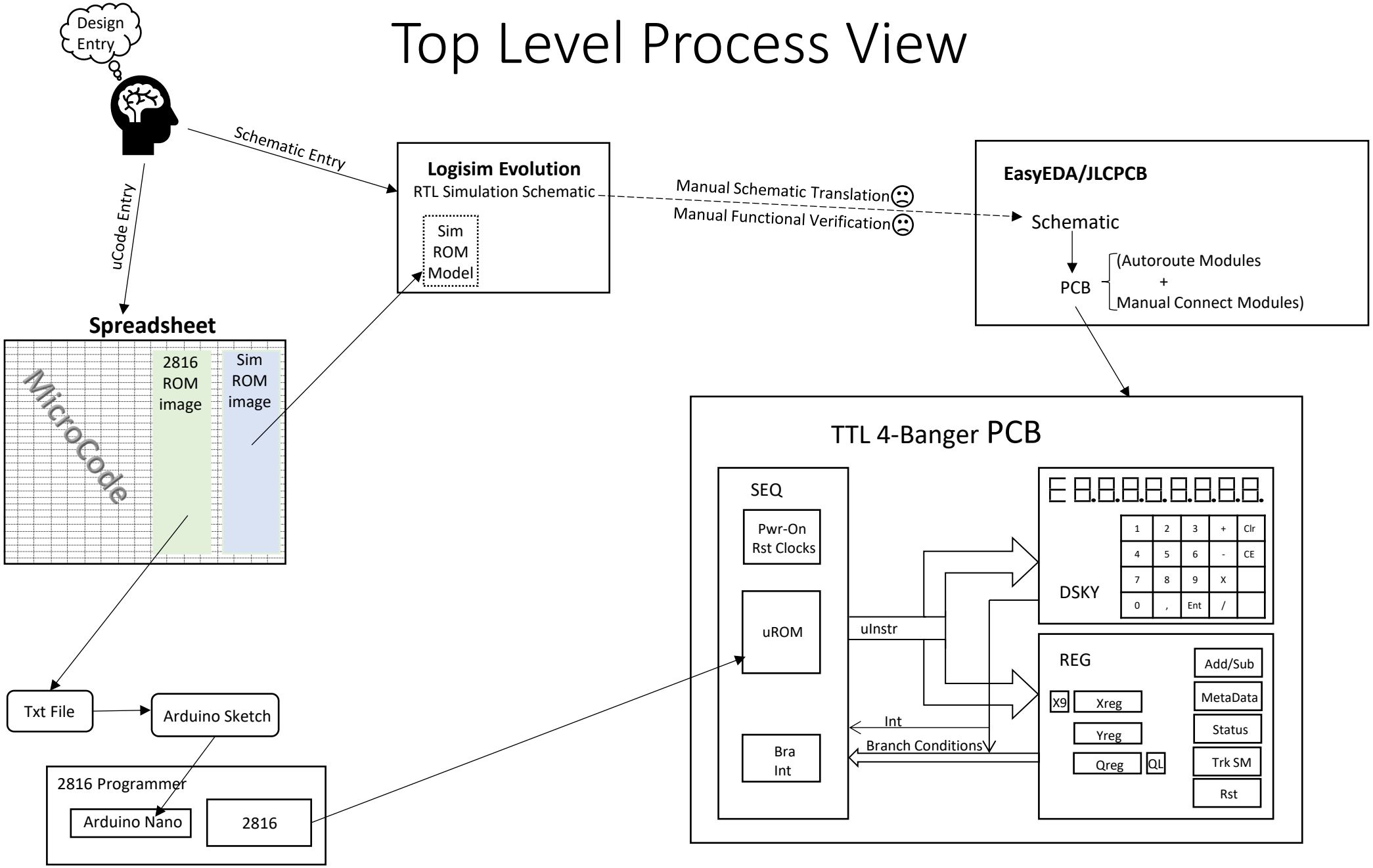
## Features and Requirements

- All 74hcXX logic implementation (plus 3 4XXX) chips, 77 total, and one 2816 EEPROM
- Purpose built architecture (not a CPU architecture)
- RPN style operation (HP Enter key)
- Add, subtract, multiply, and divide, clear entry, clear all
- 8 numeric digits plus an Error and sign digit
  - Max num: +99,999,999 to -99,999,999
  - Min Num: +0.0000001 to -0.0000001
- Floating decimal point
- Signals error on overflow for all operations, keyboard input overflow, and divide by 0
- Underflow handling
- Leading zero blanking during operand entry
- Running total operation (last result = Operand1 + Enter)
- All rising edge flip-flop design
- All on a single PCB

## 3 Major logic blocks

- SEQ: Micro-sequencer engine with one 2816 EEPROM microprogram store
  - 24-bit micro-instruction word
  - Unconditional branch, 16 conditional branches, 1 NMI-type interrupt
  - Single level loop counter (no nesting)
  - Clock and reset generation
- DSKY: Keyboard and Display unit
  - 9-digit LED display sequencing,
  - 18(20)-key keyboard encoding (2 keys not used)
- REG: Register-Arithmetic unit
  - 3, 8-digit registers (4-bits X 8)
  - 1 digit BCD adder/subtractor and exponent calculator (counters)
  - 8 status and control register bits, operation progress tracking state machine
- Architectural deficits (intentional)
  - No rounding logic
  - No guard digits
- Architectural bugs (unintentional)
  - Accuracy loss dividing 2 8-digit integers
  - Underflow condition in addition not accounted for
  - No trailing zero blanking

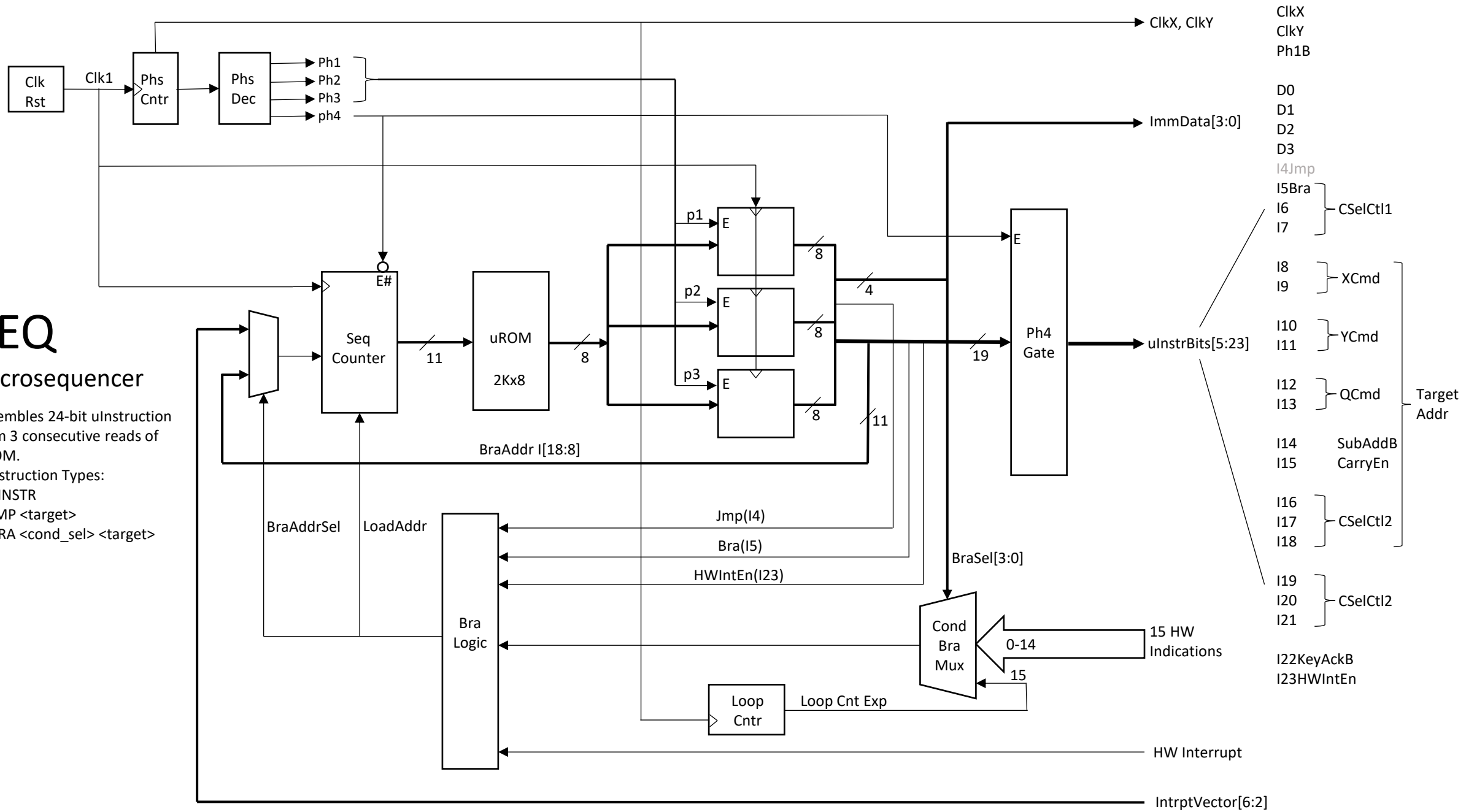
# Top Level Process View



# SEQ

## Microsequencer

Assembles 24-bit uInstruction from 3 consecutive reads of uROM.
3 Instruction Types:
- uINSTR
- JMP <target>
- BRA <cond_sel> <target>



# Sequencer (SEQ)

- Includes Clock, Reset and Timing generation.
  - Schmitt trigger reset holds sequencer in reset until power rises to ~75% VCC.
  - Schmitt trigger oscillator generates 1X clock.
  - 1X clock drives the SEQ logic.  Does not go to  external hardware.
  - 1X clock also feeds 2-bit counter which generates 4 phase signals and a divide by 4 clock.
  - Div4 clock feeds external hardware.
- Sequencer operates on a 4-phase instruction cycle to construct and present a 23-bit micro-instruction to hardware.
  - First 3 phases read three consecutive bytes from uROM and stores them in 3 8-bit registers.
  - During first 3 phases, external hardware sees 0's (inactive) on all signals.
  - Fourth phase ungates the uInstruction and presents it to external hardware.
  - D0 – D3: Immediate data and select field
  - I4:  Flow control field. This bit is reserved for the sequencer's exclusive use
    - I4=0:  Entire instruction is uINSTR type
    - I4=1:  Instruction is flow control (JMP or BRA)

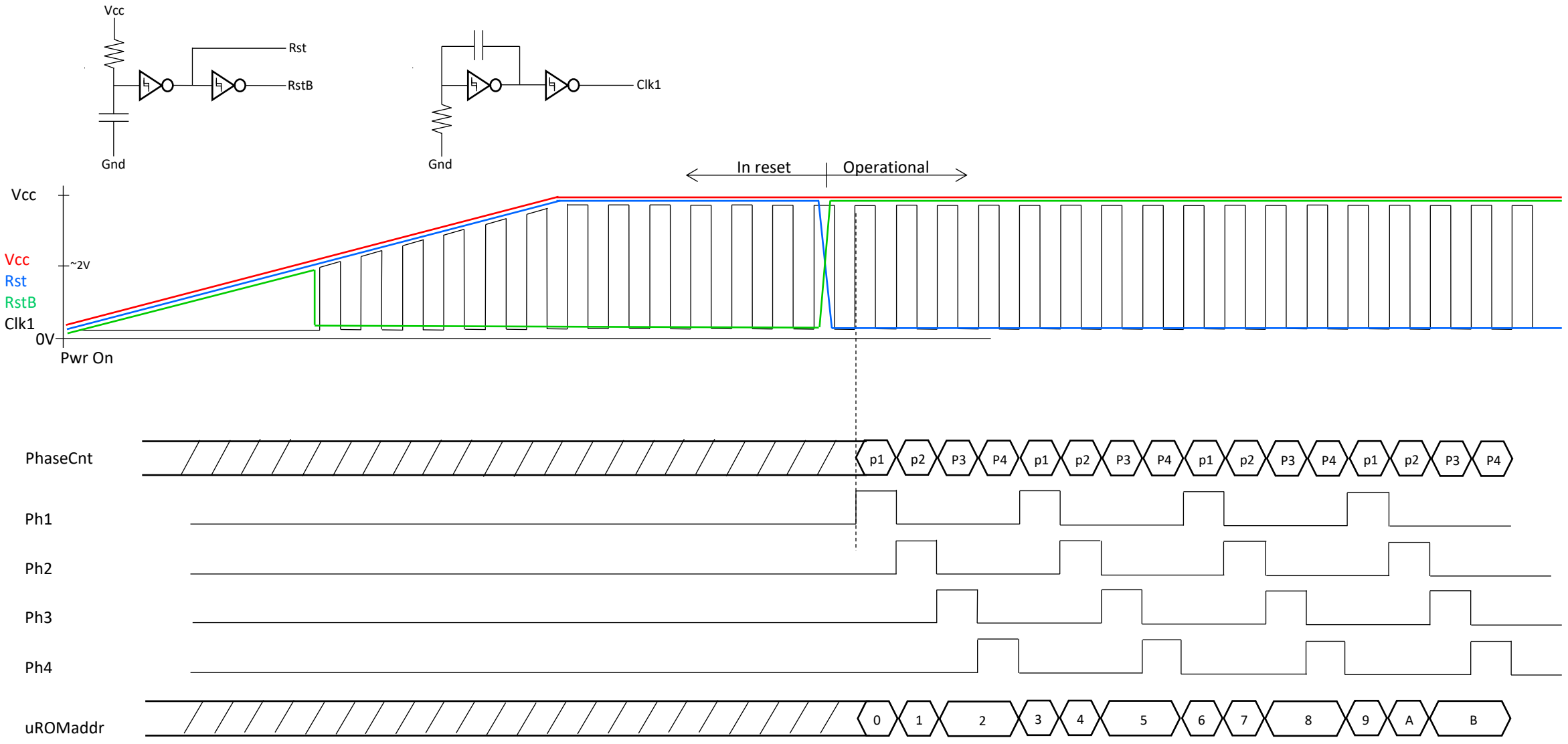
# Sequencer (SEQ)

- I5: HW control and flow control qualifier.
  - If I4=0, then I5 is just another iINSTR bit
  - If I4=1, then I5 is flow control qualifier.
    - I5=0, unconditional
    - I5=1, conditional
      - One of 16 conditions may be tested, selected by D[3:0]. If condition is true then branch is taken.
      - D[3:0]=0xF is reserved. Dedicated internally to implement a simple for-loop capability.
- I6 – I21: uINSTR bits
- I22: Loop Init: Used internal to SEQ to reset a counter that implements a simple for-loop capability.
  - I22 is also used as a uINSTR bit keeping in mind its SEQ internal use
- I23: Interrupt enable. Used internal to SEQ to enable a hardware-initiated jump to be taken if external hardware has asserted the HWInt signal when I23 asserts
  - External hardware provides the target address through a 6-bit vector, IV[6:2]
  - The target address is formed by appending IV[6:2] with b00
    - Hardware jump targets are confined to uROM addresses at 4-byte boundaries between 0x10 to 0x7C
  - I23 may also be used as a uINSTR bit keeping in mind its SEQ internal use

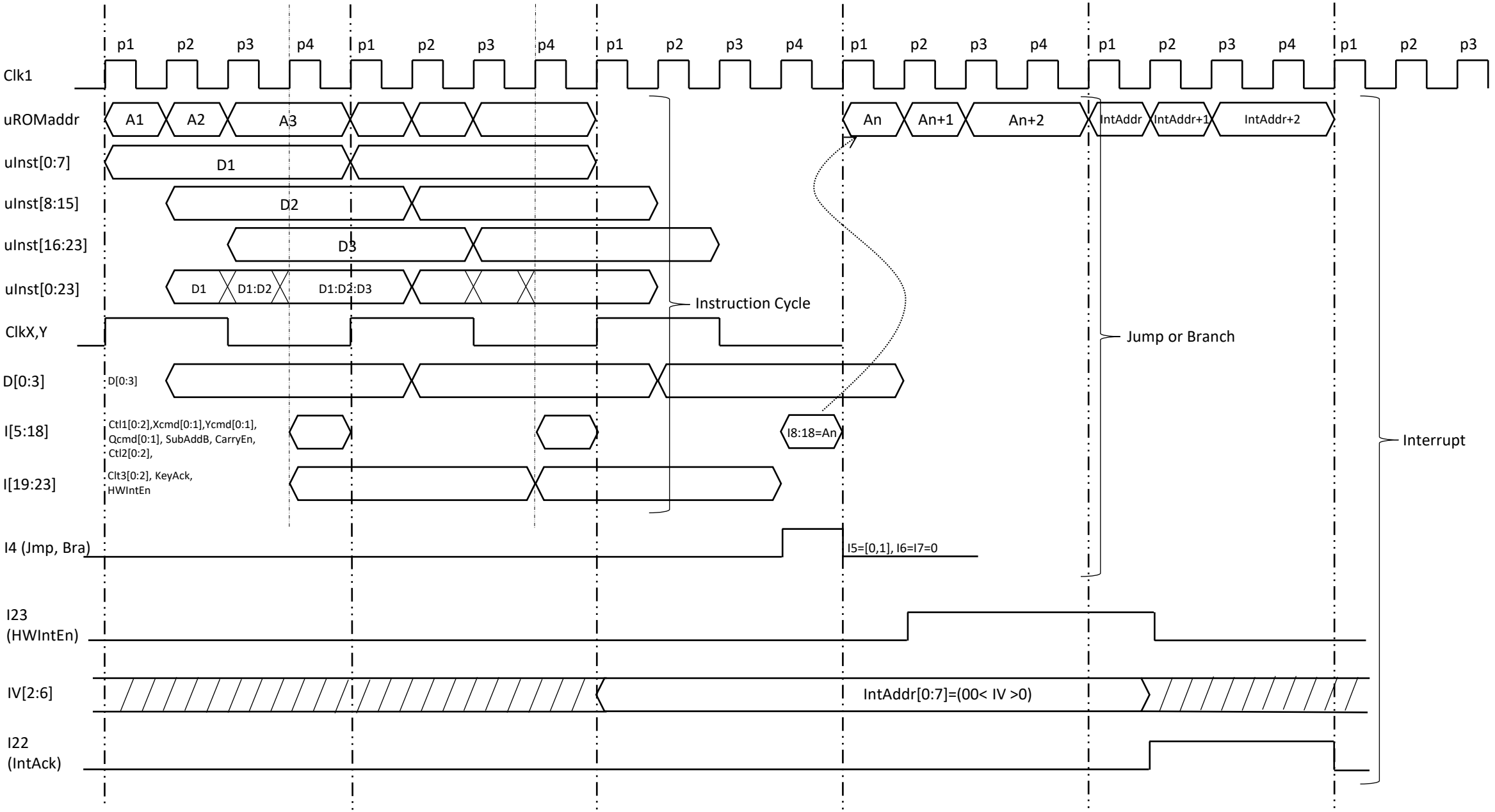
# Sequencer (SEQ)

- A primitive looping structure provides the ability to create loops up to 9 iterations (0-8)
  - Loop counter reset by asserting I22 (InitLP8)
  - Loop count tested and incremented by a conditional jump with D[3:0]=0xF (BRLoop8NZ)
  - Loop count expires when loop count reaches 8 and loop will fall through on the next loop count test (BRLoop8NZ)
- Note:  There is no subroutine call and return capability provided.
  - Since all keyboard service routines return to the keyboard-display loop, an unconditional JMP to this loop serves as an adequate return from keyboard service routines.

# Power-On Reset Startup

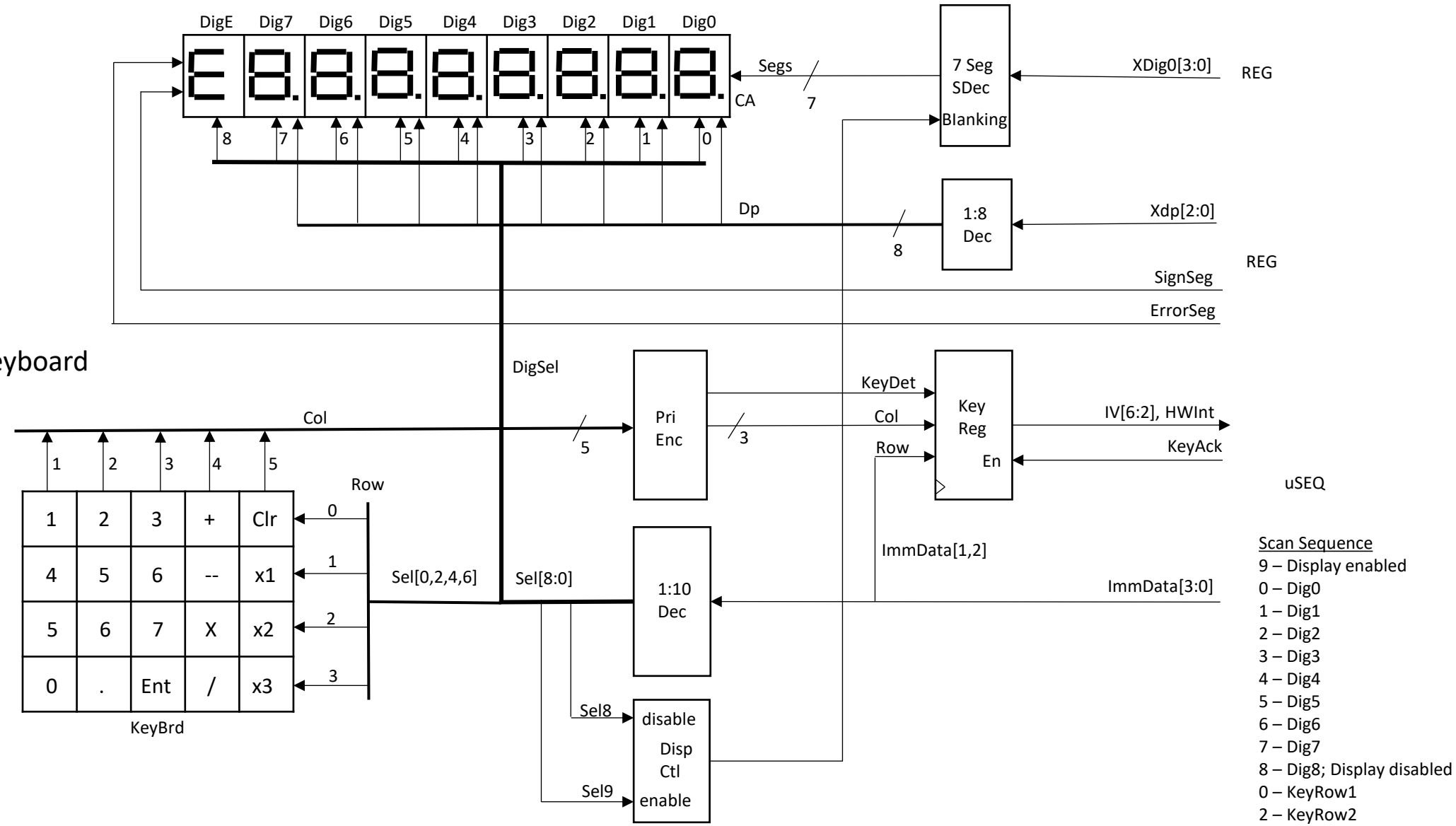


# Sequencer Instruction Timing



# DSKY

Display and Keyboard



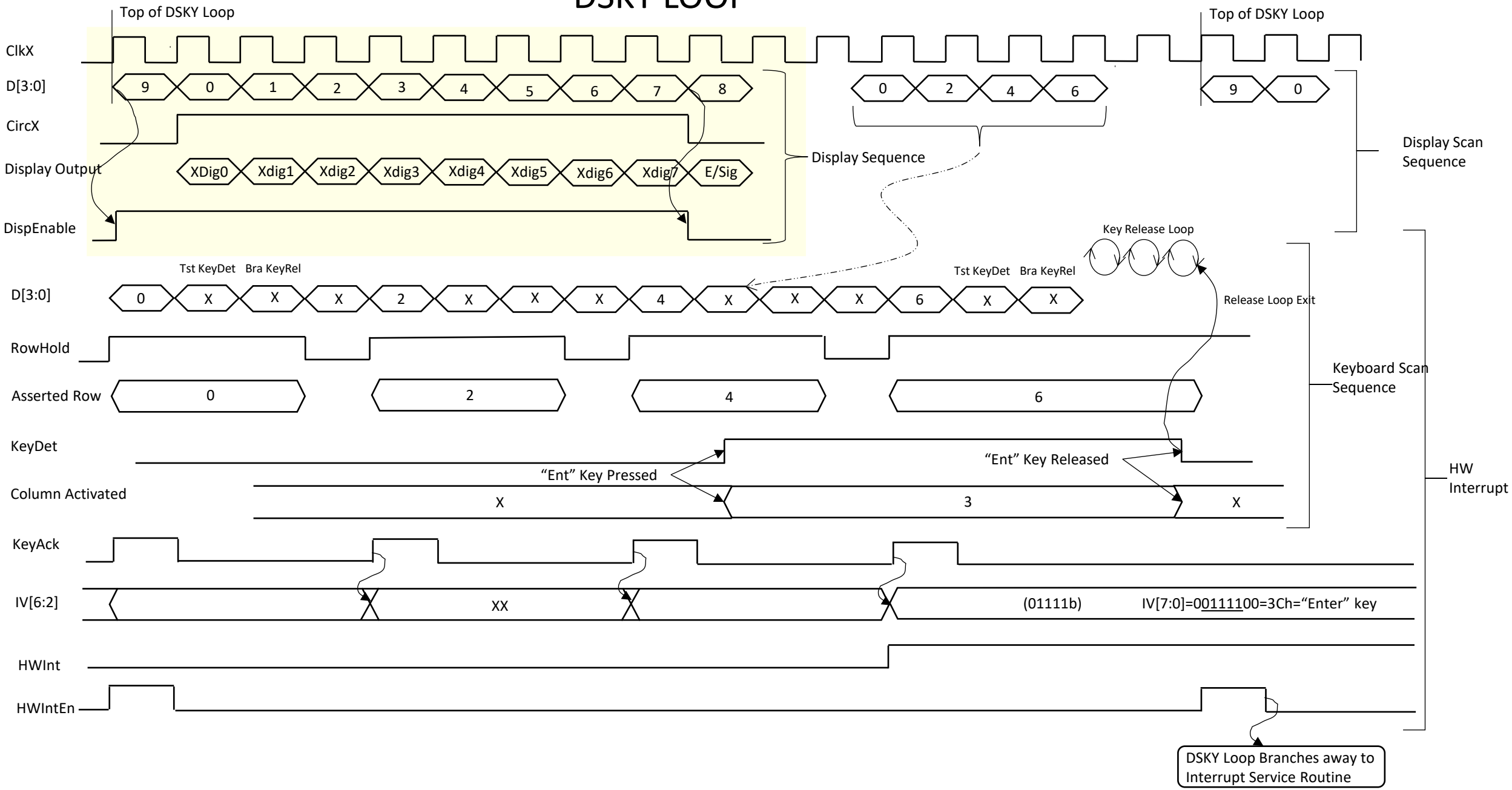

Scan Sequence

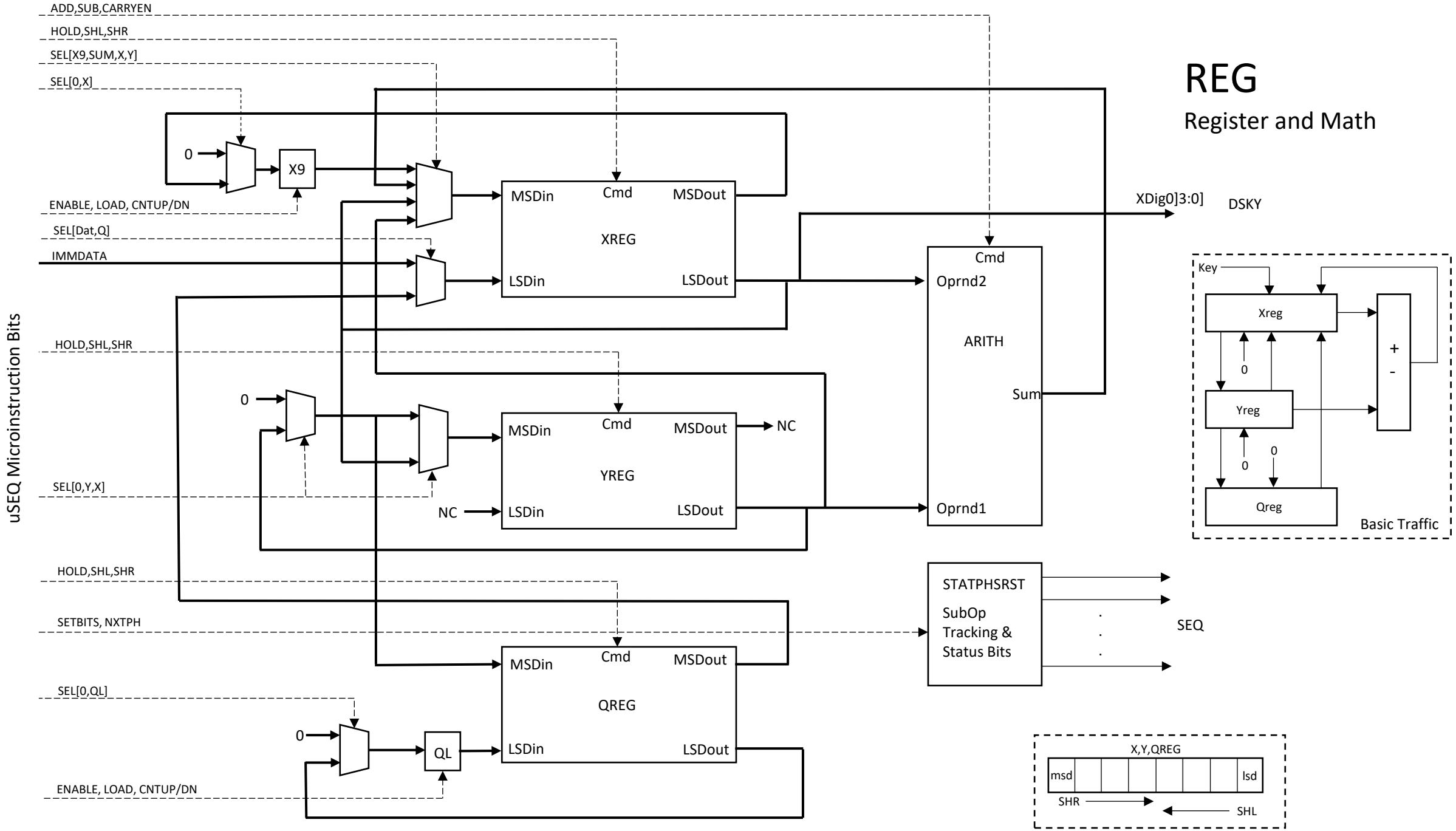
9 – Display enabled
0 – Dig0
1 – Dig1
2 – Dig2
3 – Dig3
4 – Dig4
5 – Dig5
6 – Dig6
7 – Dig7
8 – Dig8; Display disabled
0 – KeyRow1
2 – KeyRow2
4 – KeyRow3
6 – KeyRow4

# Display and Keyboard (DSKY)

- DSKY is a passive module with no autonomy.  Completely controlled by SEQ
- Display and keyboard scan is performed via D[3:0] from SEQ
  - The display is enabled by D[3:0]=b1001, and it is disabled by b1000
  - Dig0 is activated by D[3:0]=b0000, Dig7 by b0111, the Sign/Error digit by b1000
  - Row 0 of the keyboard matrix is activated by b0000, row 1 by b0010, row 2 by 0100, and row 3 by b0110.  If the display is enabled, digit 0, 2, 4, and 6, respectively, are activated with the keyboard row
- REG provides BCD data for each digit to display, 3-bit decimal point position, and the sign and error segments
- When a key is pressed in an activated row, a key detect signal is raised
- If microcode running on SEQ tests the detect signal to be true, then it may assert the KeyAck (I22) signal back to DSKY which registers the row and column where the depressed key resides and asserts the HWInt signal
- On any microcode instruction that asserts HWIntEn (I23) while HWInt is asserted, SEQ flow will jump to the microROM address formed by the concatenation of col[2:0], row[1:0], and b’00.

# DSKY LOOP


Top of DSKY Loop
Top of DSKY Loop
ClkX
D[3:0]
9 0 1 2 3 4 5 6 7 8
0 2 4 6
9 0
CircX
Display Output
XDig0 Xdig1 Xdig2 Xdig3 Xdig4 Xdig5 Xdig6 Xdig7 E/Sig
DispEnable
Display Sequence
Display Scan Sequence
Key Release Loop
Tst KeyDet Bra KeyRel
Tst KeyDet Bra KeyRel
D[3:0]
0 X X X 2 X X X 4 X X X 6 X X
Release Loop Exit
RowHold
Asserted Row
0 2 4 6
Keyboard Scan Sequence
KeyDet
"Ent" Key Released
"Ent" Key Pressed
Column Activated
X 3 X
HW Interrupt
KeyAck
IV[6:2]
XX
(01111b) IV[7:0]=00111100=3Ch="Enter" key
HWInt
HWIntEn
DSKY Loop Branches away to Interrupt Service Routine


REG
Register and Math
uSEQ Microinstruction Bits
ADD,SUB,CARRYEN
HOLD,SHL,SHR
SEL[X9,SUM,X,Y]
SEL[0,X]
0
X9
ENABLE, LOAD, CNTUP/DN
SEL[Dat,Q]
IMMDATA
MSDin
Cmd
MSDout
XREG
LSDin
LSDout
XDig0]3:0]
DSKY
Cmd
Oprnd2
ARITH
Sum
Oprnd1
HOLD,SHL,SHR
0
SEL[0,Y,X]
MSDin
Cmd
MSDout
NC
YREG
NC
LSDin
LSDout
HOLD,SHL,SHR
SETBITS, NXTPH
STATPHSRST
SubOp
Tracking &
Status Bits
SEQ
SEL[0,QL]
0
QL
MSDin
Cmd
MSDout
QREG
LSDin
LSDout
ENABLE, LOAD, CNTUP/DN
Key
Xreg
0
Yreg
0
0
Qreg
+
-
Basic Traffic
X,Y,QREG
msd
lsd
SHR
SHL

# REG: Registers, Data Steering and Compute

- REG is a passive unit with no autonomy. Completely controlled by SEQ
- Three Registers; XReg, YReg and QReg are each arrays of 8 BCD (4-bit) digits
  - Each register supports shift operations; either SHR, SHL, or both. SHR moves digits one position from MSD (most significant digit) to LSD, and the current LSD exiting the register with SRC replacing the MSD. SHL moves digits from LSD to MSD and the MSD exiting the register with SRC replacing the LSD.
    - SHRX SRC: X9, SUM, $X_{LSD}$, $Y_{LSD}$
    - SHLX SRC: IMMData, $Q_{MSD}$
    - SHRY SRC: Zero, $Y_{LSD}$, $X_{LSD}$
    - SHRQ SRC: Zero, $Y_{LSD}$
    - SHLQ SRC: Zero, $Q_{LSD}$
  - Each register may operate individually or in tandem with others but only limited cases of simultaneous register shifting are supported. Examples include: Exchange the values of Xreg and Yreg; Adding Xreg and Yreg, with SUM replacing Xreg while Yreg circulates (or is replaced with zero); Copy Xreg to Yreg while Xreg circulates (or is replaced with zero)
- X9 and QL are single BCD digit registers (loadable counters), auxiliary to Xreg and reg, respectively.
  - X9 accumulates carries out of Xmsd for Addition and Multiplication operations and is used to test Xreg digits for 0.
  - QL is used for Multiplication as a loop counter and for Division as partial quotient accumulator
  - Each can increment or decrement by one and each is loadable from two sources:
    - X9 SRC: Zero, $X_{MSD}$
    - QL SRC: Zero, $Q_{LSD}$

# Arith – Add/Sub

- Arith is a single digit BCD adder/subtractor that operates only on $X_{LSD}$ and $Y_{LSD}$. The output of Arith is the SUM routed only to $X_{MSD}$
  - Additional outputs indicate if the add generated a carry out and if the result of a subtraction is a negative number.
  - Subtraction is done through 10s complement arithmetic.
- The Add/Sub block is implemented with a cascade of three combinational logic stages:
  - 10's complement of one input
  - Summing the inputs
  - BCD correction of result
- The CarryO bit is valid for exactly one clock following the sum. This imposes a requirement for any operation on multi-digit numbers be carried out with contiguous add instructions (no intervening cycles)
- The IsNeg bit is valid for exactly one clock following the sum. This requires that action to be taken as a result of the IsNeg signal is immediate.
- If the result of a subtraction is negative, the 10's complement result is stored in Xreg and additional uCode performs subtraction of the result from 0, returning the desired result
- Arith includes 4 counters that react to key strokes to compute sum of exponents, difference of exponents, digit count, and decimal point position

# Arith – Metadata Counters

- Arith includes 4 counters that react to key strokes to compute sum of exponents, difference of exponents, digit count, and decimal point position
  - Incremented or decremented by microcode instructions
    - XDigCnt – increments for each numeric key. If zero(s) is the first keystroke, it is not counted
    - XDpCnt – increments for each numeric key after the decimal point (counts digits in fractional portion of operand)
    - SexpCnt, DexpCnt – hold the sum and difference, respectively, of the exponents of the two operands
      - Positive Exponent - first key of operand is numeric. Increment for each numeric key until decimal point is keyed
      - Negative Exponent - first key of operand is decimal point key. Decrement for each leading zero keyed until non zero numeric key
      - SexpCnt
        - for each operand, positive exponents increment up, negative exponents decrement down
      - DexpCnt
        - for first operand, positive exponents increment up, negative exponents decrement down
        - For second operand, negative exponents increment up, positive exponents decrement down
- Via microcode instructions, DexpCnt can be transferred to SexpCnt. SexpCnt can be transferred to XDpCnt. DexpCnt can be transferred to XDpCnt in two steps

# StatPhsRst

- There are two reset domains for sequential logic in REG, X and Y.
  - The Xreg domain includes Xreg and state associated with Xreg.
    - X reset is triggered by a microcode instruction and by the transition from entering the first operand to entering the second operand
  - The Yreg domain includes Yreg and Qreg and associated state.
    - Y reset is triggered by a microcode instruction
  - The Rst part of StatPhsRst is a reset synchronizer to filter spurious logic glitches from SEQ on X and Y domain resets
- The Stat part of StatPhsRst is an array of status bits to store sign bits for operands and exponents and other state information needed to parse keystrokes.  Bits are set and cleared by microcode instructions and by reset events

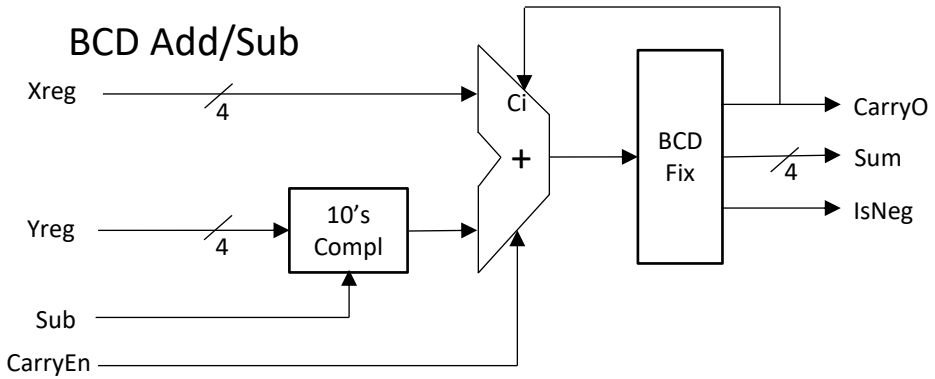
# StatPhsRst

- The Phs part of StatPhsRst is a state machine that tracks the phase of the operation underway.  Its state variables are tested by branch instructions
- States include
  - Idle* – System has cycle through reset but no keys have been pressed (virtual state)
  - Op1Ph* – First operand is being entered (virtual state)
  - EOp1Ph – First operand is entered but second operand has not started
  - Op2Ph – Second operand is being entered
  - OprPh – One of the 4 operations is underway
  - MultDivPh – One of either the mult or div operation is underway

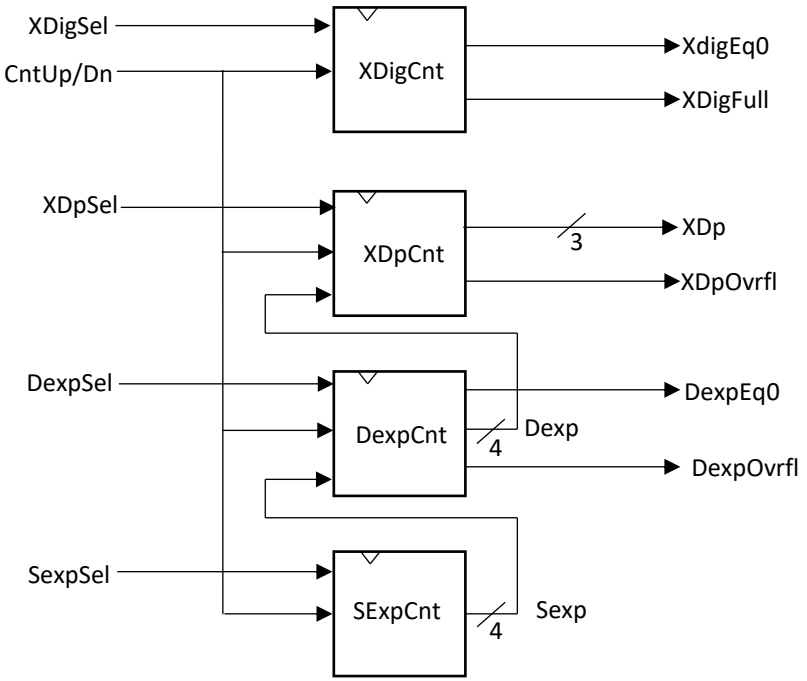
  * Idle state and Op1Ph state do not need actual state variables but they represent operational states of the overall machine
- The state machine is advanced by a microcode instruction
  - The transition from EndOp1 to Op2Ph will also happen with the first key starting operand2

# Arith

## BCD Add/Sub

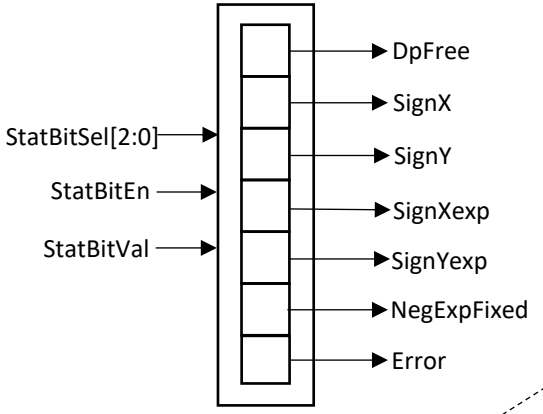



## MetaData Counters

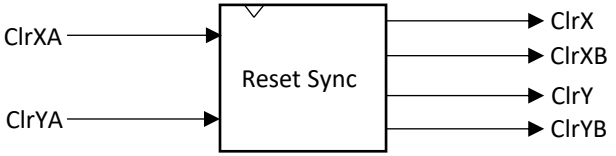



# StatPhsRst

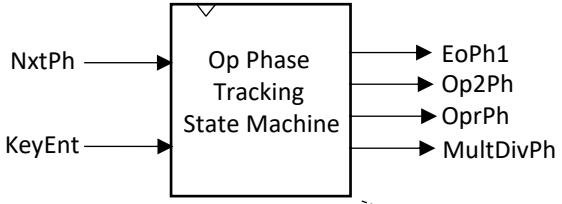
## Status Bits



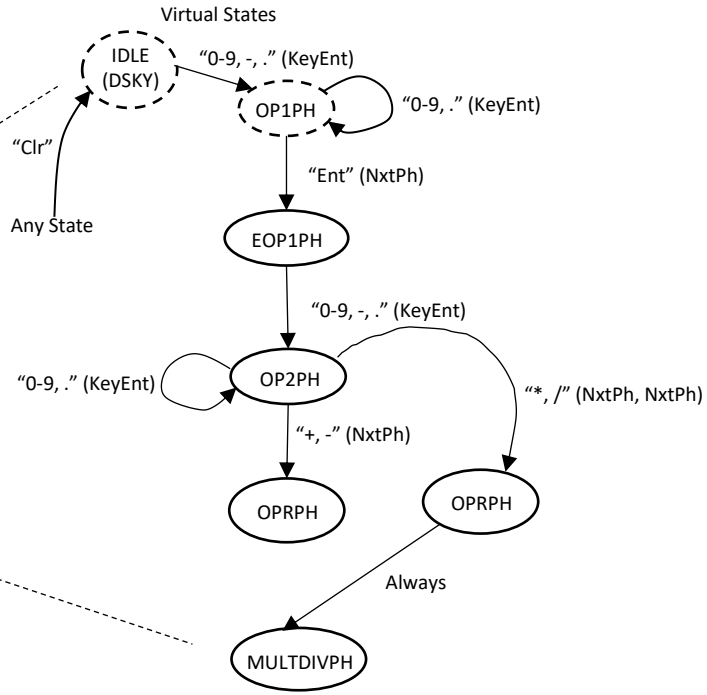

## Rst




## Phase SM

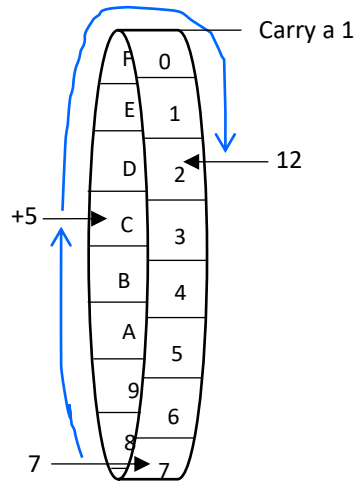



## Op Phase Tracking SM



# BCD Addition

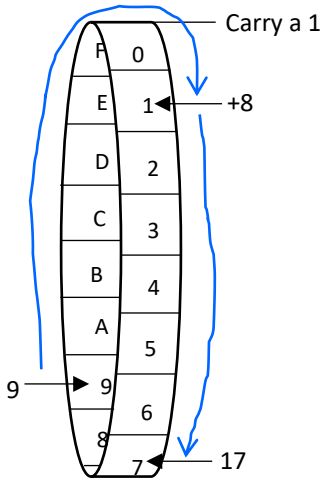
Decimal addition translated to base 16 wheel:




A+B; A, B in {0..9] BCD
Sum in [0..15]Hex -> [0..18]BCD

```
  7
+ 5
-----
0xC       If Sum>9 Then
+ 6          Add 6 to Sum
-----
12 bcd
```

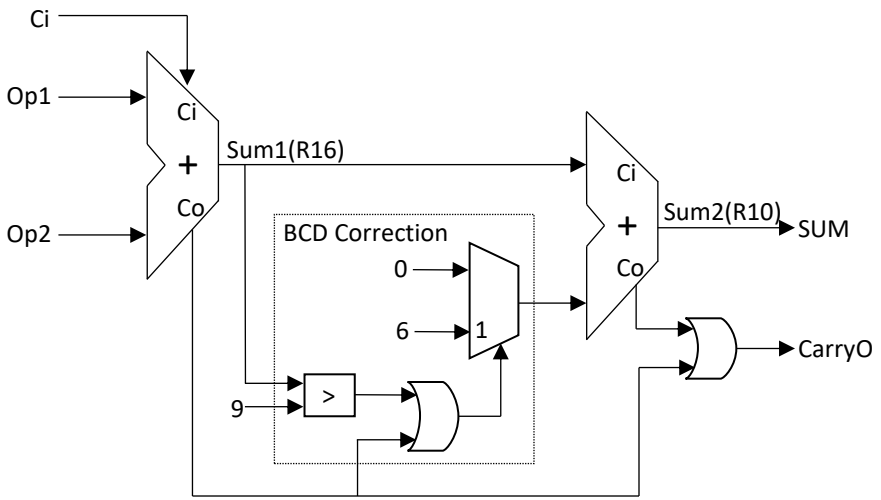
Decimal addition translated to base 16 wheel:



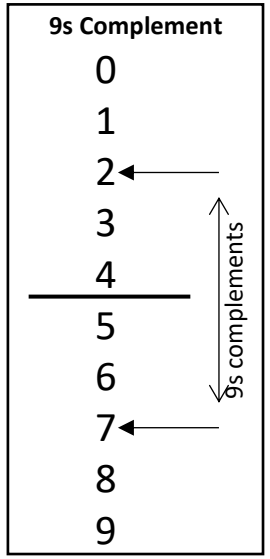

```
  9
+ 8
------
0x11      If Hex Rollover
 + 6          Add 6 to Sum
------
17 bcd
```

| | | Co | | Ci | Co | | Ci | Co | | Ci | Co | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 7693 | | | 0111 | 1 | | 0110 | 1 | | 1001 | | | 0011 | addend |
| +9684 | | | +1001 | | | +0110 | | | +1000 | | | +0100 | addend |
| 17377 | | 1 | 0001 | | | 1101 | | 1 | 0001 | | | 0111 | sum1 |
| | | | +0110 | | | +0110 | | | +0110 | | | +0000 | bdc correction |
| | | 1 | 0111 | | 1 | 0011 | | | 0111 | | | 0111 | sum2 |

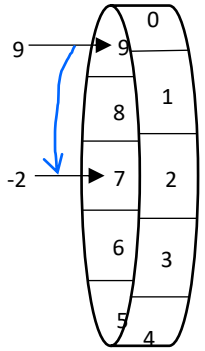
| Op1+Op2+Ci | Co | Sum1 | C | SUM |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 2 | 0 | 2 | 0 | 2 |
| 3 | 0 | 3 | 0 | 3 |
| 4 | 0 | 4 | 0 | 4 |
| 5 | 0 | 5 | 0 | 5 |
| 6 | 0 | 6 | 0 | 6 |
| 7 | 0 | 7 | 0 | 7 |
| 8 | 0 | 8 | 0 | 8 |
| 9 | 0 | 9 | 0 | 9 |
| 10 | 0 | A | 1 | 0 |
| 11 | 0 | B | 1 | 1 |
| 12 | 0 | C | 1 | 2 |
| 13 | 0 | D | 1 | 3 |
| 14 | 0 | E | 1 | 4 |
| 15 | 0 | F | 1 | 5 |
| 16 | 1 | 0 | 1 | 6 |
| 17 | 1 | 1 | 1 | 7 |
| 18 | 1 | 2 | 1 | 8 |
| 19 | 1 | 3 | 1 | 9 |



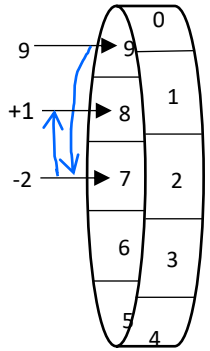
# 10’s Complement



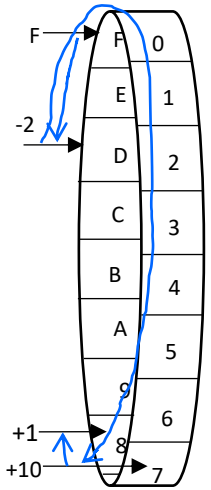

9s Complement: 9-2




10s Complement: (9-2)+1




10s Complement translated to base 16 wheel: (F-2)+10+1




For an n digit decimal number $D_nD_{n-1}…D_1D_0$;
The 10s complement is the 9s complement plus 1;
Which is:
$\{9-D_i$; for each $D_i$ (i in 0 to n)$\} + 1$

This is the same as:
$\{(0xF-D_i+10) \bmod 16$; for each $D_i$ (i in 0 to n)$\} + 1$

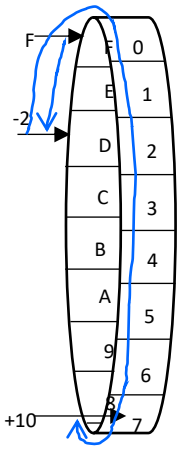
This is the same as:
$\{(1’sComp(D)+10) \bmod 16$; ; for each $D_i$ (i in 0 to n)$\} + 1$

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9 | 9 | 9 | | F | F | F | | |
| -2 | -9 | -0 | | -2 | -9 | -0 | | |
| 7 | 0 | 9 | 9s comp | D | 6 | F | | |
| + | | 1 | | +A | +A | +A | | |
| 7 | 1 | 0 | 10s comp | 7 | 0 | 9 | Mod 16 | |
| | | | | | + | 1 | | |
| | | | | | | 710 | | |

⇔

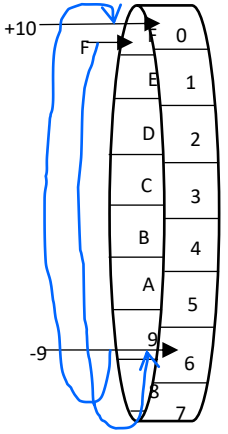
**10’s complement of 290**

9-2 = (F-2+10)mod16 = [1sComp(2)+A]mod16



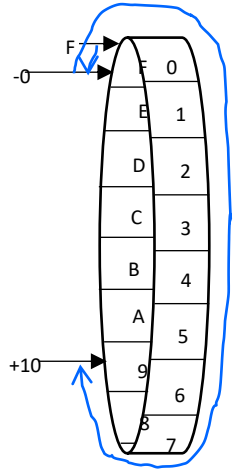

7

F-9+10 = 1sComp(9) + 10




0

F-0+10 = 1sComp(0) + 10




9

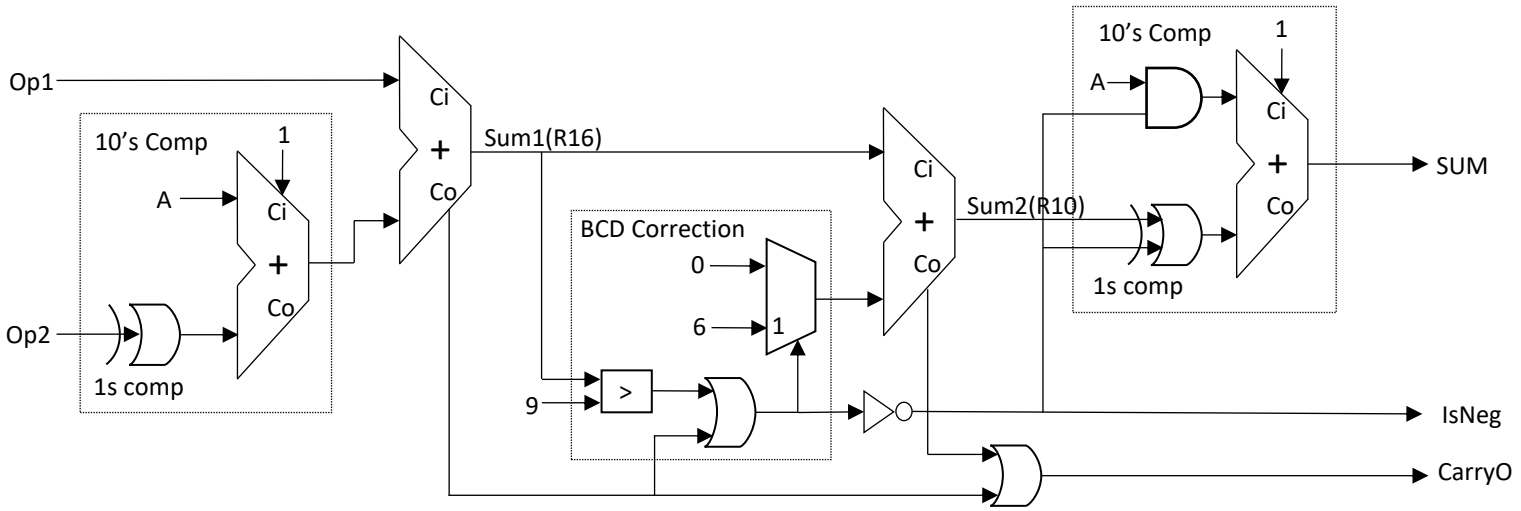
709
+ 1
---
710 = 10’s complement of 290

# BCD Subtraction

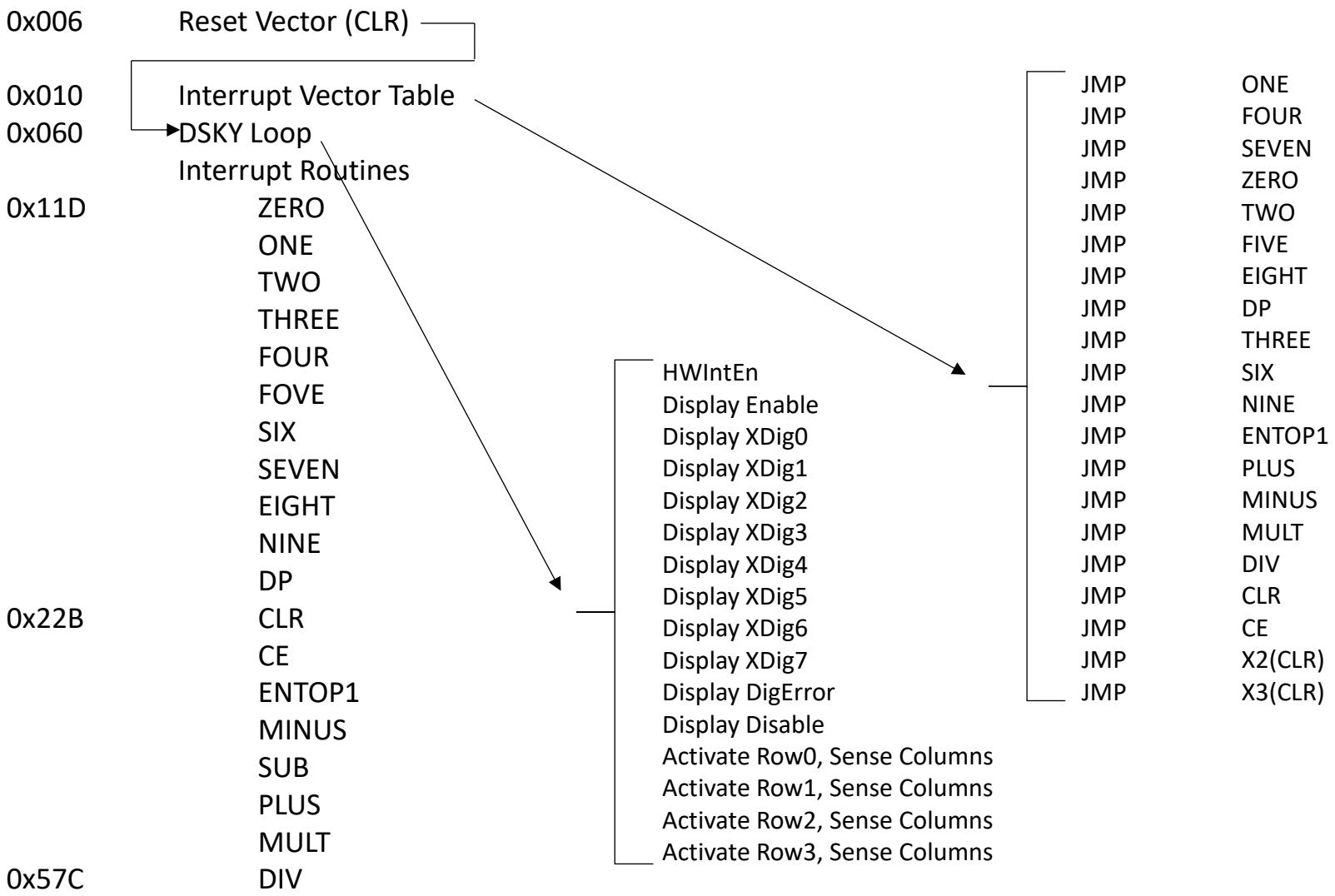
| | | Co | | Ci | Co | | Ci | |
|---|---|---|---|---|---|---|---|---|
| 47 | | | 0100 | | | 0111 | | minuend |
| -28 | | | -0010 | | | -1000 | | subtrahend |
| 19 | | | | | | | | |
| | | | 1101 | | | 0111 | | 1's complement of subtrahend |
| | | | +1010 | | | +1010 | | add 10 |
| | | 1 | 0111 | | 1 | 0001 | | 9's complement of subtrahend |
| | | | +0100 | | | +0111 | 1 | add minuend + 1 |
| | | | 1011 | | | 1001 | | difference |
| | | | +0110 | | | +0000 | | bcd correction |
| | | 1 | 0001 | | | 1009 | | |
| | | Final carry=1 difference is pos | | | | | | |

| | | Co | | Ci | Co | | Ci | |
|---|---|---|---|---|---|---|---|---|
| 28 | | | 0010 | | | 1000 | | minuend |
| -41 | | | -0100 | | | -0001 | | subtrahend |
| 13 | | | | | | | | |
| | | | 1011 | | | 1110 | | 1's complement of subtrahend |
| | | | +1010 | | | +1010 | | add 10 |
| | | 1 | 0101 | 1 | 1 | 1000 | | 9's complement of subtrahend |
| | | | +0010 | | 1 | +1000 | 1 | add minuend + 1 |
| | | 0 | 1000 | | | 0001 | | difference |
| | | | +0000 | | | +0110 | | bcd correction |
| | | | 1000 | | | 0111 | | |
| | | Final carry=0 difference is neg | | | | | | |
| | | | 0111 | | | 1000 | | 1's complement |
| | | | +1010 | | | +1010 | | add 10 |
| | | | 0001 | | 1 | 0010 | | 9's complement |
| | | | +0000 | | | +0000 | 1 | add 1 |
| | | | 0001 | | | 0011 | | |

| Op1-Op2 | Co | A + 10#(B) | SUM |
|---|---|---|---|
| -9 | 0 | 1 | -9 |
| -8 | 0 | 2 | -8 |
| -7 | 0 | 3 | -7 |
| -6 | 0 | 4 | -6 |
| -5 | 0 | 5 | -5 |
| -4 | 0 | 6 | -4 |
| -3 | 0 | 7 | -3 |
| -2 | 0 | 8 | -2 |
| -1 | 0 | 9 | -1 |
| 0 | 0 | A | 0 |
| 1 | 0 | B | 1 |
| 2 | 0 | C | 2 |
| 3 | 0 | D | 3 |
| 4 | 0 | E | 4 |
| 5 | 0 | F | 5 |
| 6 | 1 | 0 | 6 |
| 7 | 1 | 1 | 7 |
| 8 | 1 | 2 | 8 |
| 9 | 1 | 3 | 9 |



# MicroCode Organization



# MicroCode Sample

| 7.5.3 | | | I[5:0], I23 1[5:0], 3[7] | 2[7:0], 3[2:0] | I[3:0] 1[3:0] | I[22] 3[6] | I[13:8] 2[5:0] | I[7:5] 1[7:5] | I[18:16] 3[2:0] | I[21:19] 3[5:3] | I[3:0] 1[3:0] | I[14] 2[6] | I[15] 2[7] | Logisim uROM Image D[7:0] I[15:8] [23:16] | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Addr** | **Hex** | **Label** | **SEQ** | | **DSKY** | | **REG** | | | | | **AddSub** | | **1[7:0]** | **2[7:0]** | **3[7:0]** | **4** |
| | | | **Bra** | **Target** | **Sel[3:0]** | **KeyAck** | **RegCmd** | **SelCtl1** | **SelCtl2** | **SelCtl3** | **KeyMCntSel** | **Sub_AddB** | **CarryEn** | v2.0 raw | | | Required for Logisim |
| 675 | 2A3 | **ENTOP1** | BRSignXexp | SETOP1DPFIR | | | | | | | | | | 3E | A9 | 02 | If Op1 has neg exponent (DPFirst) then save in SignYexp ( |
| 678 | 2A6 | | JMP | TSTDIGEMPTY | | | | | | | | | | 10 | AC | 02 | Otherwise goto test for empty X reg |
| 681 | 2A9 | SETOP1DPFIR | | | | | | | StatBitEn | StatBit1 | SignYexp | | | 05 | 00 | 1B | |
| 684 | 2AC | TSTDIGEMPTY | BRDigEmpty | ENTERZERO | | | | | | | | | | 32 | B5 | 02 | If no digit hit yet, Jump to enter0 |
| 687 | 2AF | | BRDPFree | ADJEXPFIXDP | | | | | | | | | | 37 | BE | 02 | If DP was never hit then goto adjust exp and fixDP |
| 690 | 2B2 | | JMP | LEFTJUST | | | | | | | | | | 10 | C7 | 02 | Otherwise go to left justify |
| 693 | 2B5 | ENTERZERO | | | | | | | CntrEn | CntrIncr | XDigCnt | | | 01 | 00 | 36 | Incr Xreg Dig count (enter a 0) |
| 696 | 2B8 | | | | | | | | StatBitEn | StatBit1 | FixDP | | | 00 | 00 | 1B | and fix the DP |
| 699 | 2BB | | JMP | LEFTJUST | | | | | | | | | | 10 | C7 | 02 | then left justify |
| 702 | 2BE | ADJEXPFIXDP | | | | | | | CntrEn | | XYSexpCnt | | | 04 | 00 | 06 | If X is whole integer, fix DP and adjust exponent |
| 705 | 2C1 | | | | | | | | CntrEn | | XYDexpCnt | | | 03 | 00 | 06 | |
| 708 | 2C4 | | | | | | | | StatBitEn | StatBit1 | FixDP | | | 00 | 00 | 1B | |
| 711 | 2C7 | LEFTJUST | BRDigFull | COPYXY | | | | | | | | | | 31 | D6 | 02 | Left justify X until Xdig is full |
| 714 | 2CA | | | | | | LeftJustX | SelXData | | | 0 | | | 00 | 02 | 00 | Shift left with 0's in at msd |
| 717 | 2CD | | | | | | | | CntrEn | CntrIncr | XDigCnt | | | 01 | 00 | 36 | and adjust Xdp and Xdig count to match |
| 720 | 2D0 | | | | | | | | CntrEn | CntrIncr | XDPCnt | | | 02 | 00 | 36 | |
| 723 | 2D3 | | JMP | LEFTJUST | | | | | | | | | | 10 | C7 | 02 | |
| 726 | 2D6 | COPYXY | | | | InitLp8 | | | | | | | | 00 | 00 | 40 | Copy Xreg to Yreg |
| 729 | 2D9 | | BRLoop8NZ | COPYXYE | | | | | | | | | | 3F | DC | 02 | |
| 732 | 2DC | COPYXYE | | | | | CopyXY | SelXReg | SelXXlsd | SelYXlsd | | | | 20 | 05 | 28 | |
| 735 | 2DF | | BRLoop8NZ | COPYXYE | | | | | | | | | | 3F | DC | 02 | |
| 738 | 2E2 | | BRSignX | CPSIGXSIGY | | | | | | | | | | 34 | E8 | 02 | Copy Xsign to Ysign |
| 741 | 2E5 | | JMP | RMLDZEROS | | | | | | | | | | 10 | EB | 02 | |
| 744 | 2E8 | CPSIGXSIGY | | | | | | | StatBitEn | StatBit1 | SignY | | | 04 | 00 | 1B | |
| 747 | 2EB | RMLDZEROS | | | | InitLp8 | | | | | | | | 00 | 00 | 40 | Loop to remove leading zeros from fractional args. Clea |
| 750 | 2EE | BRN1E | BRLoop8NZ | RLZENTERE | | | | | | | | | | 3F | F4 | 02 | Burn loop 1 of 9 and jump to entry point |
| 753 | 2F1 | RLZTOPE | | | | | RmLdZeroX | SelXData | | | 0 | | | 00 | 02 | 00 | Shift Xmsd out the top and zero in from the bottom |

# MicroCode Fields, Mnemonics and Codes

| | Jmp/Bra | 1[4] | 1[3:0] |
|---|---|---|---|
| | JMP | 1 | 0000 |
| BR0 | BRDecEven | 1 | 0000 |
| BR1 | BRDigFull | 1 | 0001 |
| BR2 | BRDigEmpty | 1 | 0010 |
| BR3 | BRDintOvrfl | 1 | 0011 |
| BR4 | BRSignX | 1 | 0100 |
| BR5 | BRSignY | 1 | 0101 |
| BR6 | BRInOp2Ph | 1 | 0110 |
| BR7 | BRDPFree | 1 | 0111 |
| BR8 | BRQLZero | 1 | 1000 |
| BR9 | BRX9Zero | 1 | 1001 |
| BR10 | BRDpOvrfl | 1 | 1010 |
| BR11 | BRNoKeyDet | 1 | 1011 |
| BR12 | BRIsNeg | 1 | 1100 |
| BR13 | BRCarryO | 1 | 1101 |
| BR14 | BRSignXexp | 1 | 1110 |
| BR15 | BRLoop8NZ | 1 | 1111 |
| | BRSignYexp | 1 | 0110 |
| | BrNegExpTerm | 1 | 0101 |
| | ExtraBR2 | 0 | 0000 |
| | ExtraBR3 | 0 | 0000 |
| | HWINTEN | 0 | 0000 |

| RegCmd | 2[5:0] |
|---|---|
| CircX | 000001 |
| DPAdjX | 000001 |
| EntNumX | 000010 |
| LeftJustX | 000010 |
| DPAdjY | 000100 |
| CopyXY | 000101 |
| ExchXY | 000101 |
| AddSub | 000101 |
| DivXY10 | 000101 |
| ShrX9X8QDiv10 | 010001 |
| CopyYQ | 010100 |
| CopyQX | 100010 |
| ApendDivdn | 000010 |
| RmLdZeroX | 000010 |
| CopyQ0QL | 100000 |
| ShrX | 000001 |
| ShlX | 000010 |
| CopyYX | 000101 |
| ExtraCmd3 | 000000 |

| SelCtl1 | 1[7:5] |
|---|---|
| SelXReg | 001 |
| LoadX9 | 010 |
| SelXQmsd | 011 |
| LoadQL | 100 |
| SHDexpXDP | 101 |
| Reserved1 | 110 |
| Reserved2 | 111 |
| HWINTEN | 000 |
| SelXData | 000 |
| SelXSumX9 | 000 |
| ExtraSelCtl1 | 000 |
| ExtraSelCtl2 | 000 |
| ExtraSelCtl3 | 000 |
| ExtraSelCtl4 | 000 |

| | |
|---|---|
| BR* | 001 |

| SelCtl2 | 3[2:0] |
|---|---|
| ClrXPh1 | 001 |
| ClrYPh2 | 010 |
| StatBitEn | 011 |
| SelXYlsd | 100 |
| SelXSum | 100 |
| NxtPh | 101 |
| CntrEn | 110 |
| SHSexpDexp | 111 |
| SelXX9 | 000 |
| SelXXlsd | 000 |
| ExtraSelCtl1 | 000 |
| ExtraSelCtl2 | 000 |

| ALUCmd | 2[7:6] |
|---|---|
| SubXY | x1 |
| CarryEn | 1x |

| CtlSel3 | 3[5:3] |
|---|---|
| SelQLQlsd | 001 |
| SelYQYlsd | 010 |
| StatBit1 | 011 |
| SelX9Xmsd | 100 |
| SelYXlsd | 101 |
| CntrIncr | 110 |
| RowHold | 111 |
| SelYZero | 000 |
| SelQZero | 000 |
| SelX9Zero | 000 |
| SelQLZero | 000 |
| ExtraSelCtl3 | 000 |

| SEQ/DSKY | 3[7:6] |
|---|---|
| HWINTEN | 10 |
| KeyAck | 01 |
| InitLP8 | 01 |

| KeyMCntSel | 1[3:0] |
|---|---|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |
| 6 | 0110 |
| 7 | 0111 |
| 8 | 1000 |
| 9 | 1001 |
| BlankNum | 1111 |
| XDigCnt | 0001 |
| XDPCnt | 0010 |
| XYDexpCnt | 0011 |
| XYSexpCnt | 0100 |
| X9Cnt | 0101 |
| QLCnt | 0110 |
| FixDP | 0000 |
| SignXexp | 0001 |
| NegExpTerm | 0010 |
| SignX | 0011 |
| SignY | 0100 |
| SignYexp | 0101 |
| AssertError | 0110 |

**ImmData asserts on Ph2 of each cycle**

**SelCtl1, SelCtl2, RegCmd, ALUCmd assert on Ph4 and is valid only Ph4**

**SelCtl3, KeyAckB, HWIntEn Assert on Ph4 of each cycle**

**SelCtl3 Can assert simutaneous with JMP/BR*. SelCtl1/2 cannot.**

# MicroInstruction Fields

### SelCtl1 (I5br:I7)

| | | |
|---|---|---|
| SelXReg | 001 | Select X reg MSD source from X reg or Y reg group |
| LoadX9 | 010 | Load X9 source into X9 reg |
| SelXQmsd | 011 | Select X reg source from Q Dig7 |
| LoadQL | 100 | Load QL source into QL reg |
| SHDexpXDP | 101 | Load Dexp counter value into XDP counter |
| Reserved1 | 110 | Currently unassigned |
| Reserved2 | 111 | Currently unassigned |
| SelXData | 000 | Select X LSD reg source from D[3:0] |
| SelXSumX9 | 000 | Select X reg MSD source from Sum or X9 group |

### SelCtl2 (I16:I18)

| | | |
|---|---|---|
| SelQLQlsd | 001 | Select QL reg source from Q reg LSD |
| SelYQYlsd | 010 | Select Y and Q reg MSD sources from Y reg LSD |
| StatBit1 | 011 | Set selected status bit to 1 |
| SelX9Xmsd | 100 | Select X9 reg source from X reg MSD |
| SelYXlsd | 101 | Select Y reg source from X reg LSD |
| CntrIncr | 110 | Increment selected and enabled counter |
| RowHold | 111 | DSKY row latch disable (hold) |
| SelYZero | 000 | Select Y reg MSD source from 0 |
| SelQZero | 000 | Select Q reg MSD source from 0 |
| SelX9Zero | 000 | Select X9 reg source from 0 |
| SelQLZero | 000 | Select QL reg source from 0 |

### SelCtl3 (I19:I21)

| | | |
|---|---|---|
| ClrXPh1 | 001 | Clear X reg domain devices |
| ClrYPh2 | 010 | Clear Y reg domain devices |
| StatBitEn | 011 | Enable status latches for write |
| SelXYlsd | 100 | Select X reg MSD source from Y reg LSD |
| SelXSum | 100 | Select X reg MSD source from SUM |
| NxtPh | 101 | Advance tracking state machine state |
| CntrEn | 110 | Enable selected counter for up/down advance |
| SHSexpDexp | 111 | Load Sexp counter value into Dexp |
| SelXX9 | 000 | Select X reg MSD source from X9 reg |
| SelXXlsd | 000 | Select X reg MSD source from X reg LSD |

### RegCmd (I8:I13)

| | | |
|---|---|---|
| CircX | 000001 | Shift X reg from MSD toward LSD |
| DPAdjX | 000001 | |
| ShrX | 000001 | |
| EntNumX | 000010 | Shift X reg from LSD toward MSD |
| LeftJustX | 000010 | |
| ApendDivdn | 000010 | |
| RmLdZeroX | 000010 | |
| DPAdjY | 000100 | Shift X reg from MSD toward LSD |
| CopyXY | 000101 | Shift X and Y reg from MSD toward LSD |
| ExchXY | 000101 | |
| AddSub | 000101 | |
| DivXY10 | 000101 | |
| ShrX9X8QDiv10 | 010001 | Shift X and Q reg from MSD toward LSD |
| CopyYQ | 010100 | Shift Y and Q reg from MSD toward LSD |
| CopyQ0QL | 100000 | Shift Q reg from LSD toward MSD |
| CopyQX | 100010 | Shift X and Q reg from LSD toward MSD |

### Branches (D3:D0)

| | | |
|---|---|---|
| JMP | NA | Unconditional Branch |
| BRDecEven | 0000 | Branch if DexpCnt=0 |
| BRDigFull | 0001 | Branch of DigCnt=8 |
| BRDigEmpty | 0010 | Branch if DigCnt=0 |
| BRDintOvrfl | 0011 | Branch if Dexp>=8 |
| BRSignX | 0100 | Branch if SignX status bit set |
| BRSignY | 0101 | Branch if SignY status bit set |
| BRInOp2Ph | 0110 | Branch if in or beyond Op2Ph tracking state |
| BRDPFree | 0111 | Branch of DP status bit has not been set |
| BRQLZero | 1000 | Branch if Qlreg=0 |
| BRX9Zero | 1001 | Branch if X9reg=0 |
| BRDpOvrfl | 1010 | Branch if XDpCnt>=8 |
| BRNoKeyDet | 1011 | Branch if no key is currently pressed |
| BRIsNeg | 1100 | Branch if arithmetic (SUM) sign bit is set |
| BRCarryO | 1101 | Branch if arithmetic carry out bit is set |
| BRSignXexp | 1110 | Branch if Op2's exponent sign status bit is set |
| BRLoop8NZ | 1111 | Branch if LoopCnt>0 |
| BRSignYexp | 0110 | Branch if Op1's exponent sign status bit is set |
| BrNegExpTerm | 0101 | Branch if current negative exponent calculated |

### Immediate Data Labels (D3:D0)

| | | |
|---|---|---|
| 0 | 0000 | Immedate data 0 |
| 1 | 0001 | Immedate data 1 |
| 2 | 0010 | Immedate data 2 |
| 3 | 0011 | Immedate data 3 |
| 4 | 0100 | Immedate data 4 |
| 5 | 0101 | Immedate data 5 |
| 6 | 0110 | Immedate data 6 |
| 7 | 0111 | Immedate data 7 |
| 8 | 1000 | Immedate data 8 |
| 9 | 1001 | Immedate data 9 |
| XDigCnt | 0001 | Select the XDigit counter |
| XDPCnt | 0010 | Select the XDP counter |
| XYDexpCnt | 0011 | Select the Dexp counter |
| XYSexpCnt | 0100 | Select the Sexp counter |
| X9Cnt | 0101 | Select the X9 counter |
| QLCnt | 0110 | Select the QL counter |
| FixDP | 0000 | Select the DP status bit |
| SignXexp | 0001 | Select the Op2 exponent sign status bit |
| NegExpTerm | 0010 | Select the negative exponent accumulation terminated status bit |
| SignX | 0011 | Select the Op2 sign status bit |
| SignY | 0100 | Select the Op1 sign status bit |
| SignYexp | 0101 | Select the Op1 exponent sign status bit |
| AssertError | 0110 | Select the Error status bit |

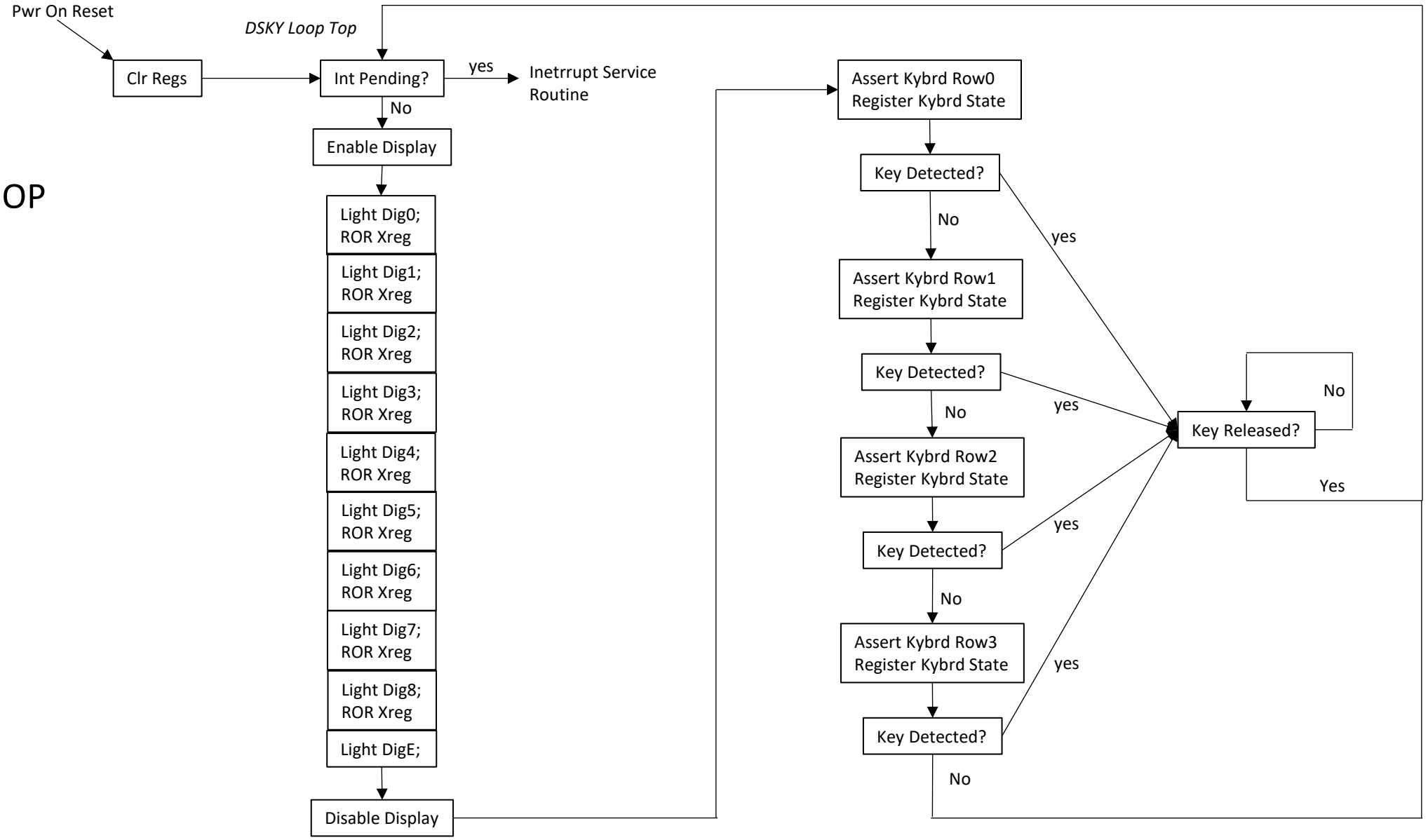
### Add/Sub (I14:I15)

| | | |
|---|---|---|
| SubAddB | 0 | Add Digit 0 of Y register with Digit 0 of X register |
| | 1 | Subtract Digit 0 of X register from Digit 0 of Y register |
| CarryEn | 0 | Disable Carry In to Adder/Subtractor for one instruction |
| | 1 | Enable Carry In to Adder/Subtractor for one instruction |

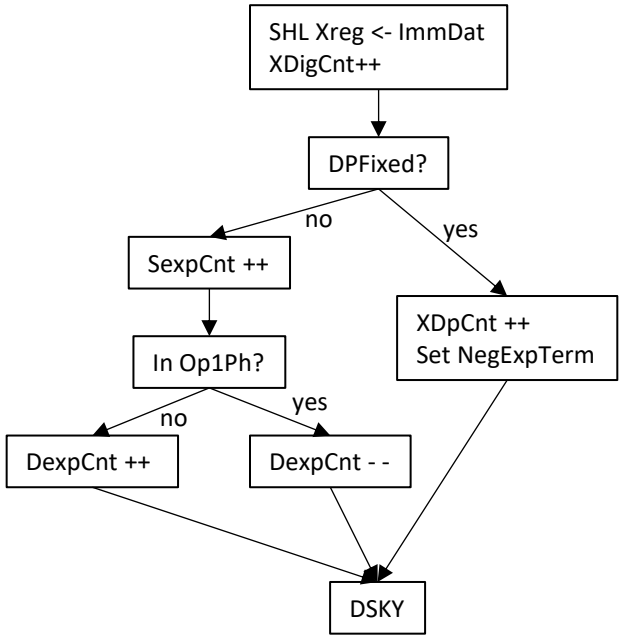
### SEQ/DSKY (I22:I23)

| | | |
|---|---|---|
| KeyAck | 0 | NA |
| | 1 | Write state of keyboard into keyboard register, reset sequencer loop counter |
| HWIntEn | 0 | NA |
| | 1 | Enable branch to interrupt service routine if HWInt is active for one instruction |

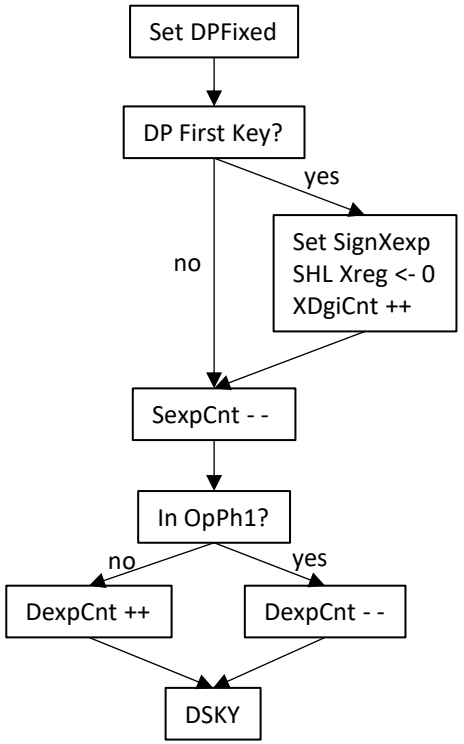
# DSKY LOOP


Pwr On Reset
DSKY Loop Top
Clr Regs
Int Pending?
yes
Inetrrupt Service Routine
No
Enable Display
Light Dig0; ROR Xreg
Light Dig1; ROR Xreg
Light Dig2; ROR Xreg
Light Dig3; ROR Xreg
Light Dig4; ROR Xreg
Light Dig5; ROR Xreg
Light Dig6; ROR Xreg
Light Dig7; ROR Xreg
Light Dig8; ROR Xreg
Light DigE;
Disable Display
Assert Kybrd Row0 Register Kybrd State
Key Detected?
No
yes
Assert Kybrd Row1 Register Kybrd State
Key Detected?
No
yes
Assert Kybrd Row2 Register Kybrd State
Key Detected?
No
yes
Assert Kybrd Row3 Register Kybrd State
Key Detected?
No
yes
Key Released?
No
Yes

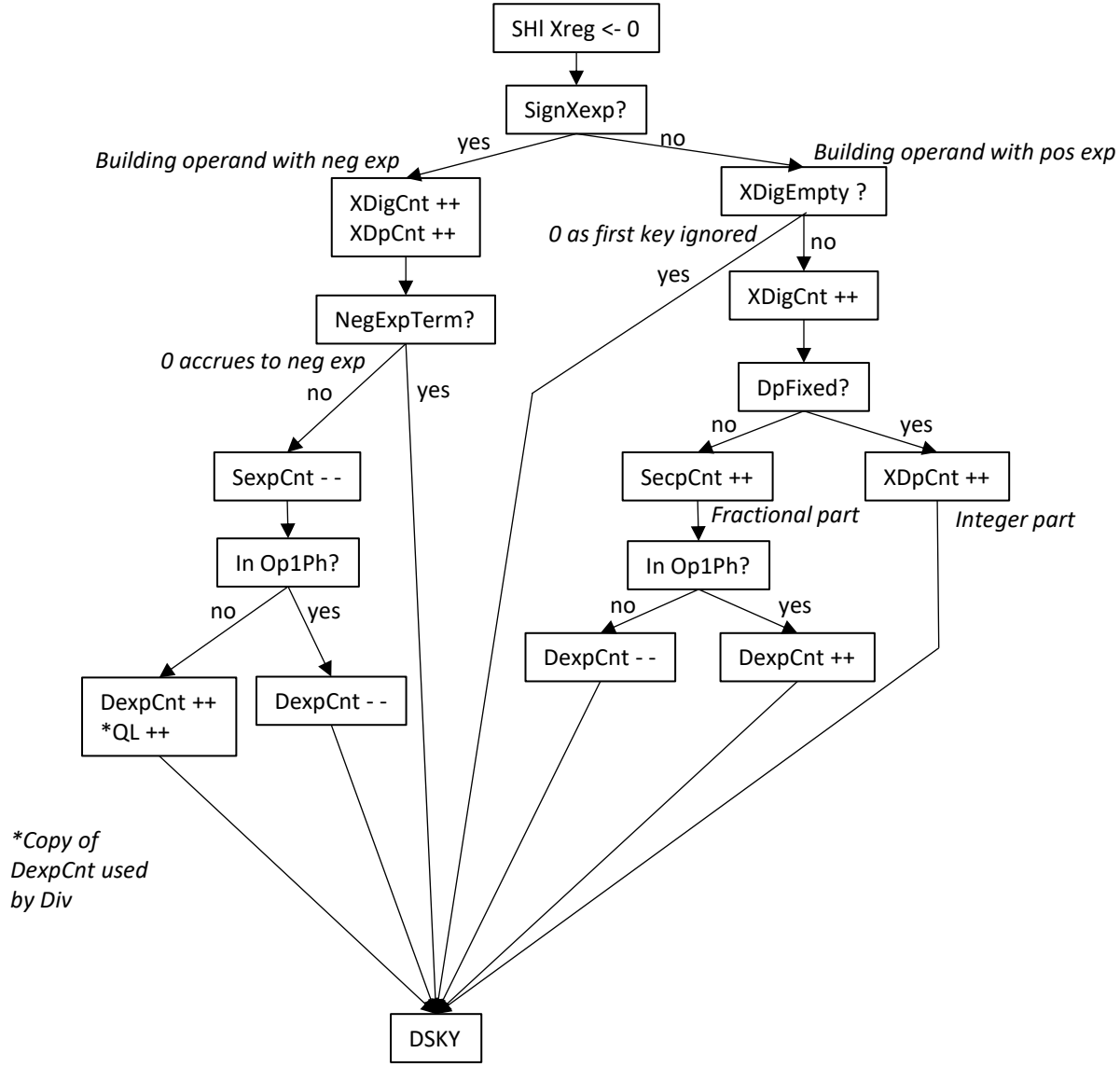
## 1-9 Service Routine



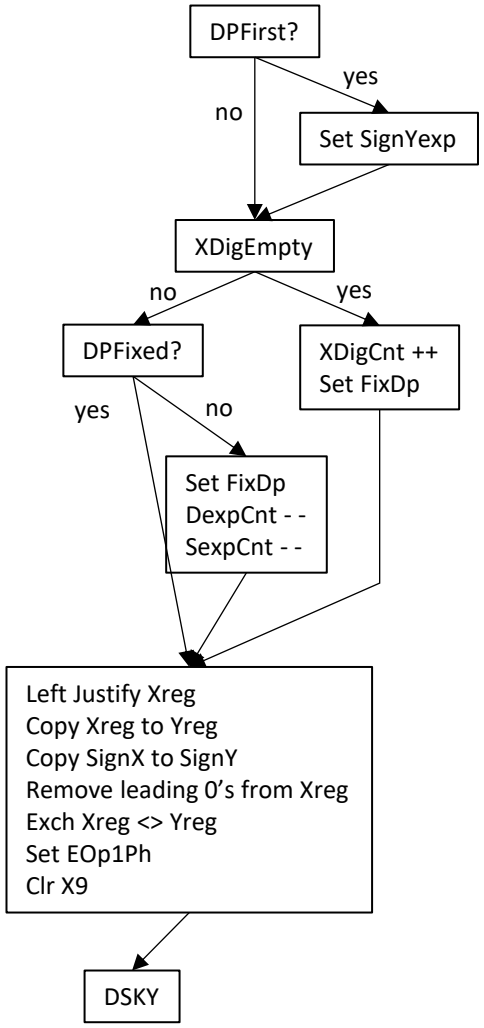

## Dp Service Routine



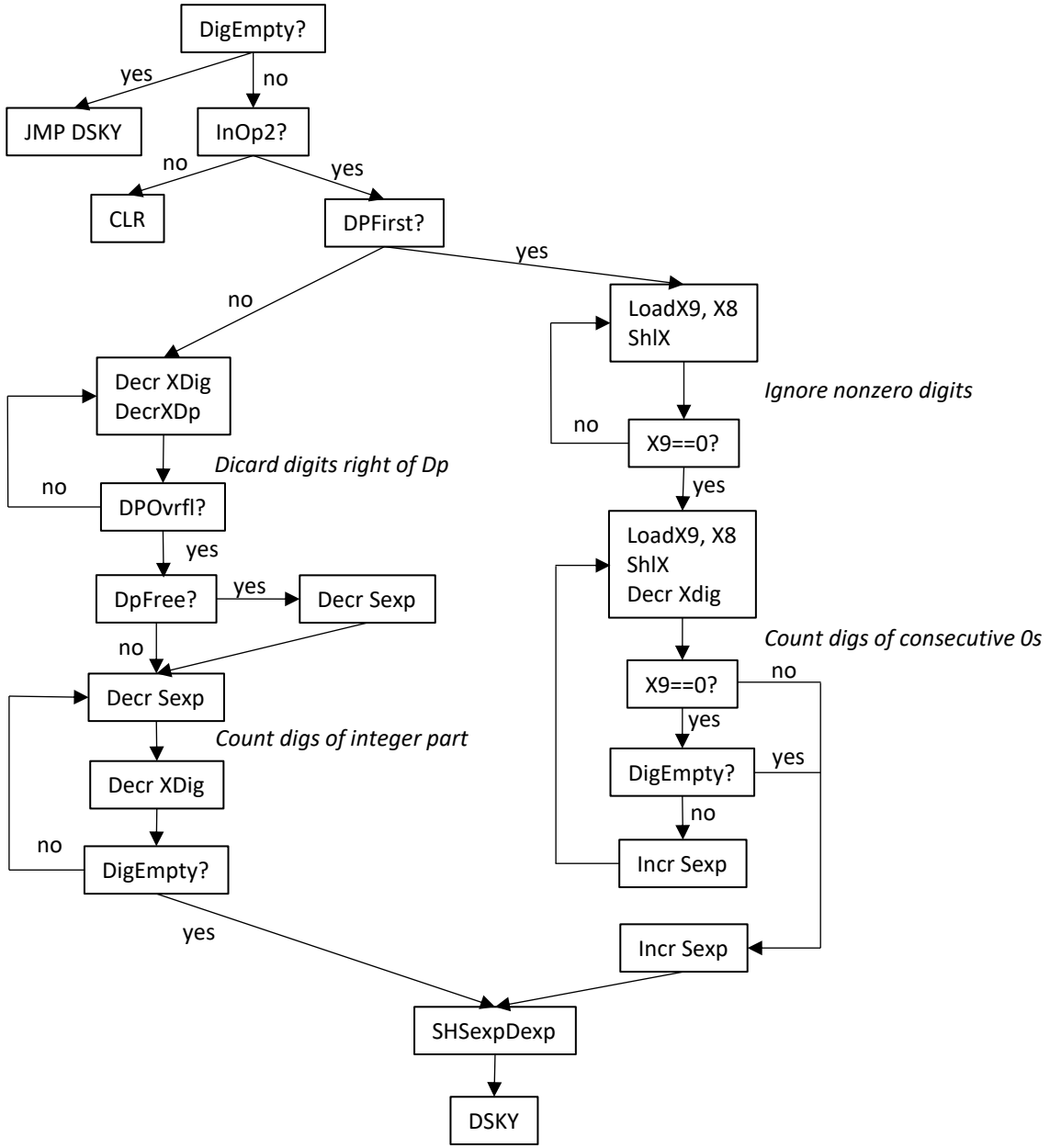

## Zero Service Routine



## Enter Op1 Service Routine



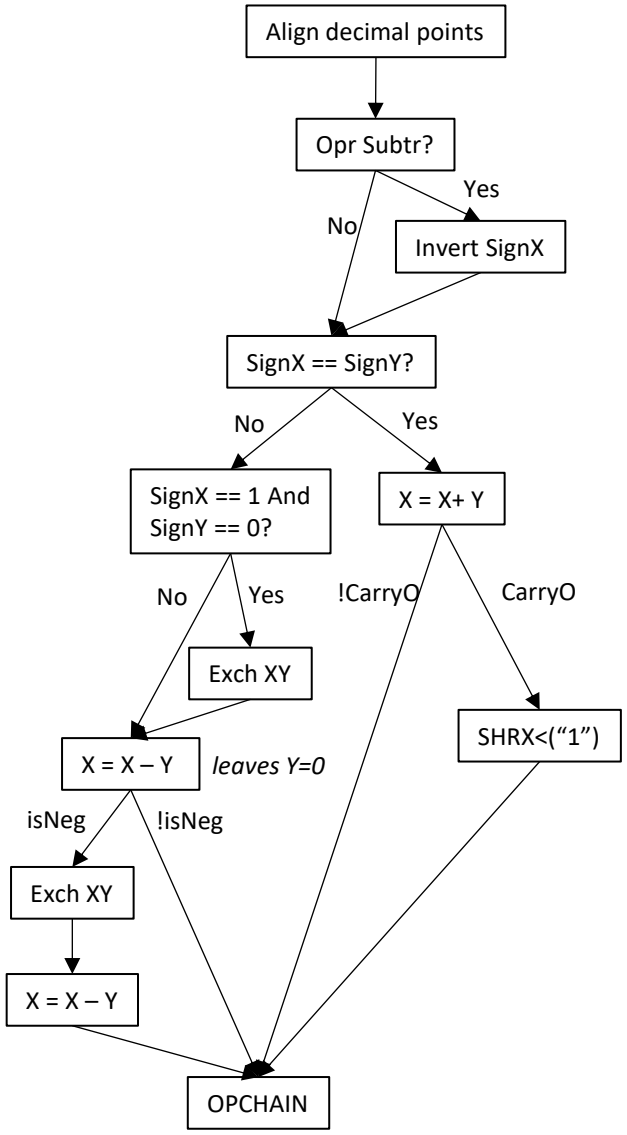

## CE Service Routine



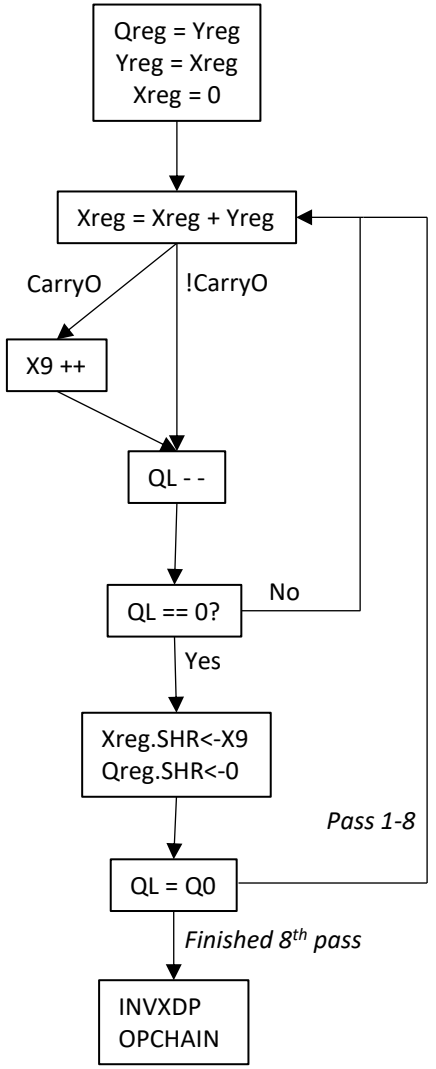
## Basic Register Model

Xreg : Array[8:1] of BCD digits
X9 : BCD digit
Yreg : Array[8:1] of BCD digits
Qreg : Array[8:1] of BCD digits
QL : BCD digit
Xdp : Integer[2:0]
DpFixed : Boolean
Sexp : Integer[3:0]
Dexp : Integer[3:0]
NegExpDPFixed : Boolean

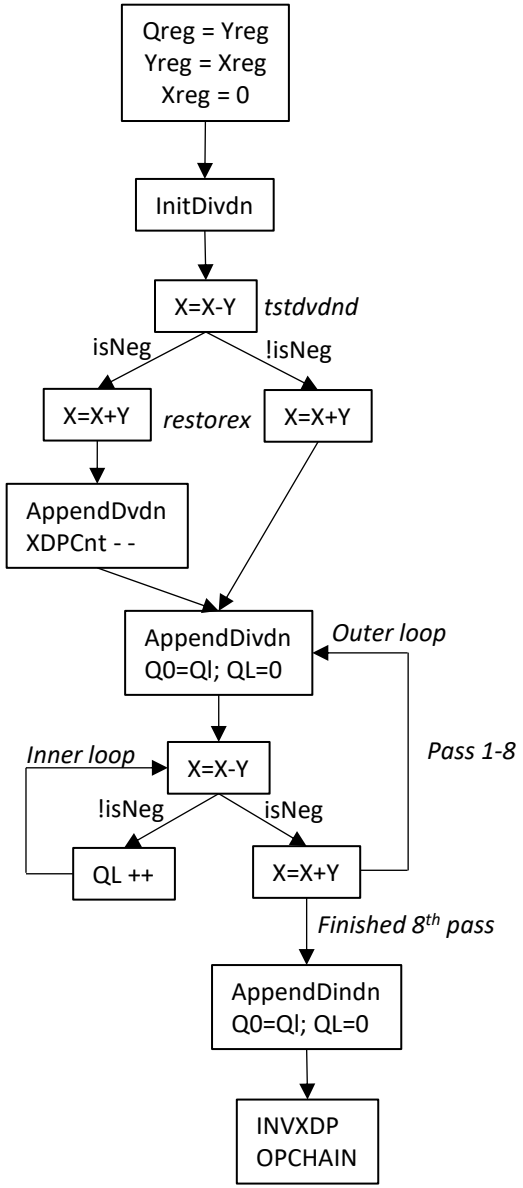
## Add or Sub



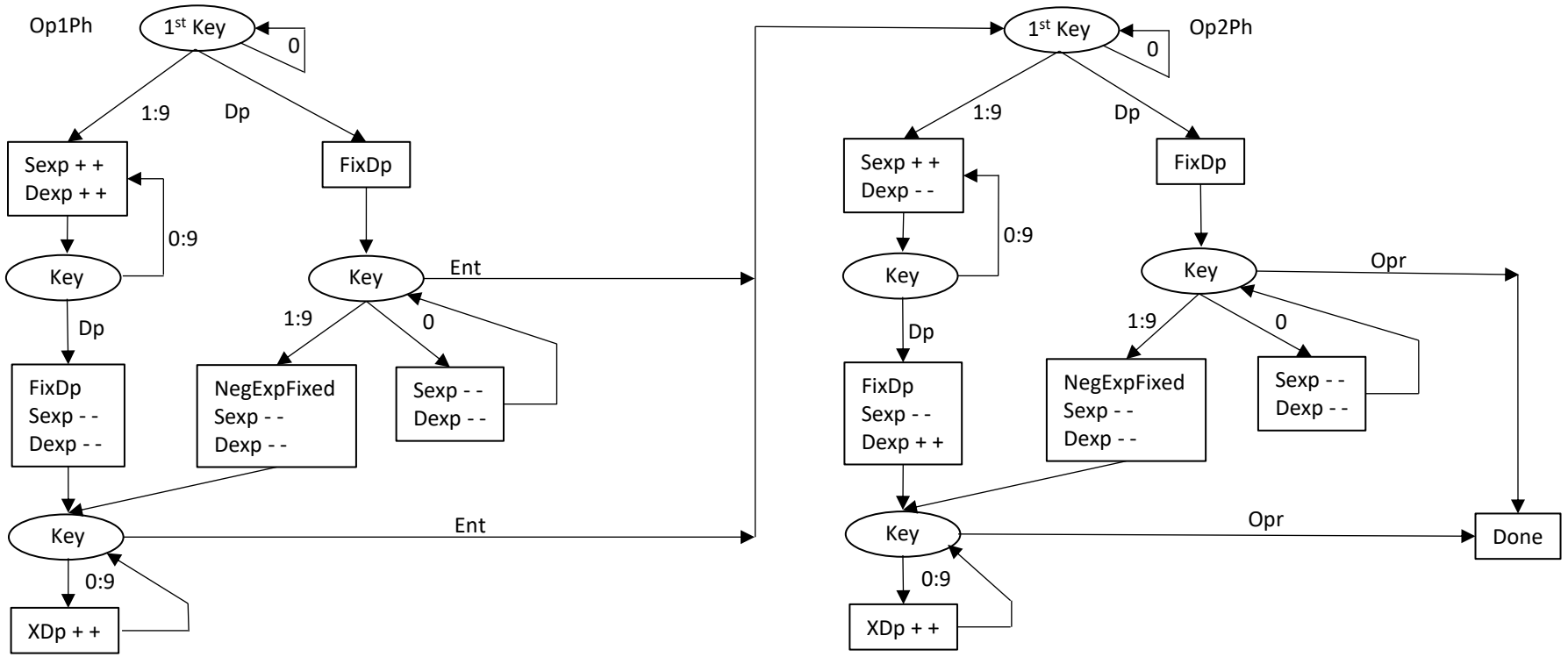

## Mult




## Div



# Exponent Computation (XDp Placement)

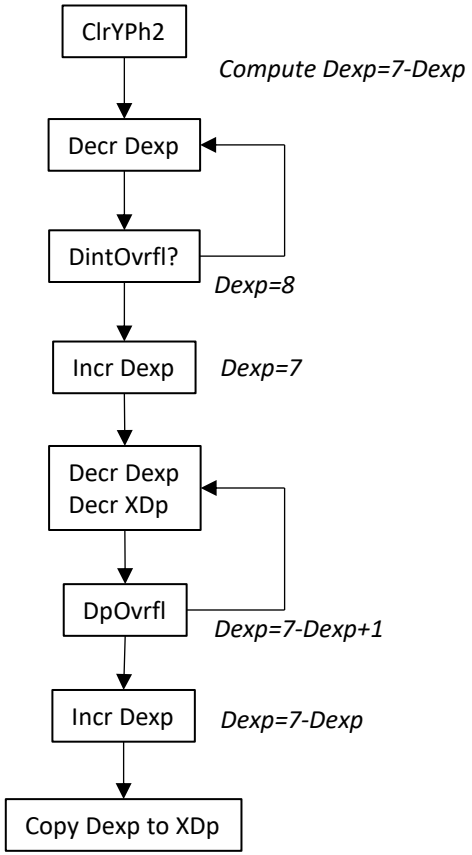



Add/Sub:
If Dexp <= 0, then XDp = XDp(Op2) else XDp = XDp(Op2) + Dexp.

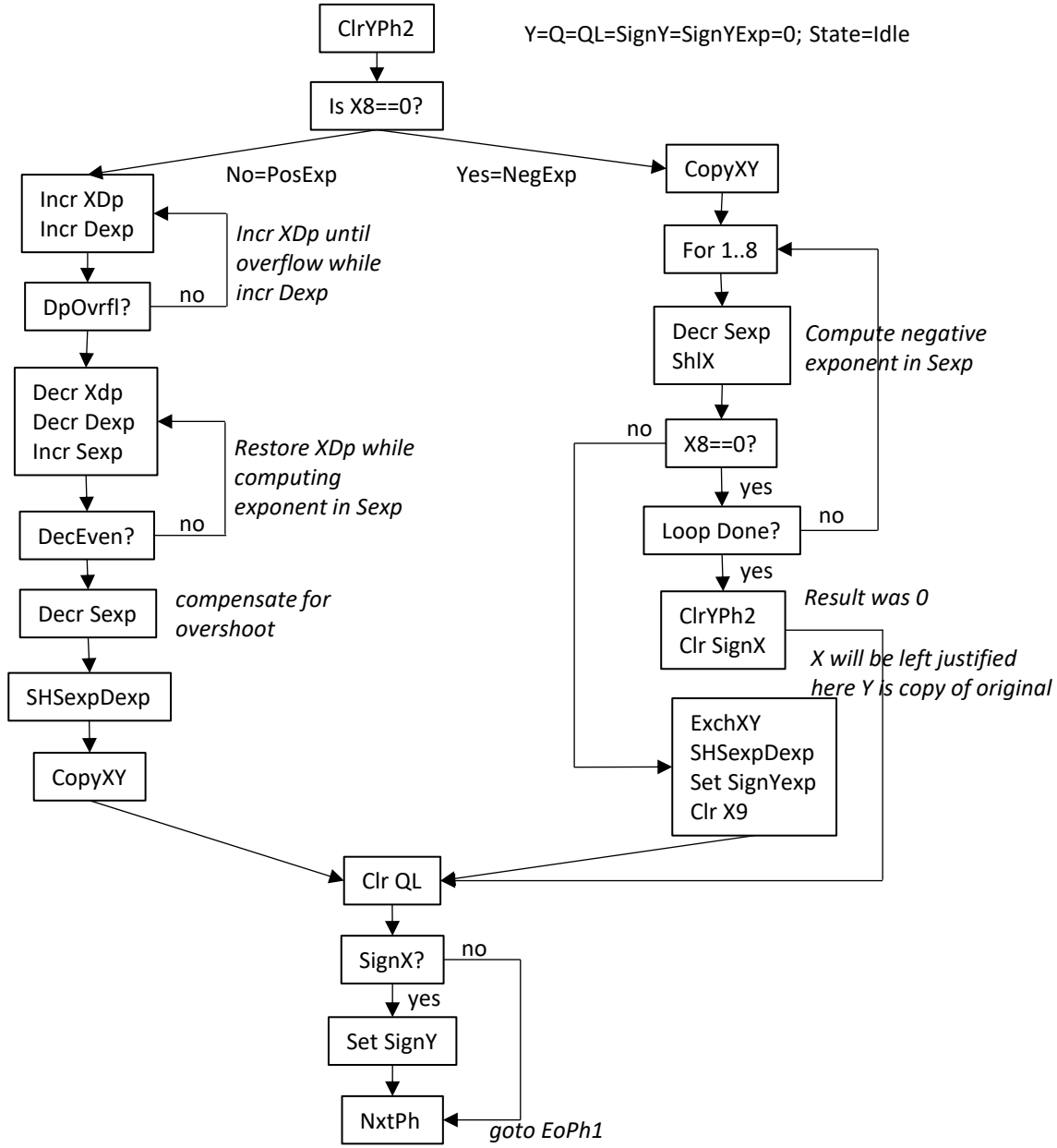
Mult
Pos Exp: XDp placement is initialized by Sexp. Incremented once (XDp shifted right one digit) if the final Mult loop leaves a non-zero accumulation of CarryOs in X9.
Neg Exp: XDp placed in left most position. Sexp number of leading zeros right shifted into MSD of product, losing right most digits for each shift.

Div:
Pos Exp: XDp placement is initialized by Dexp. Decremented once (XDp shifted left one digit) if the minimum number of dividend digits needed to subtract the divisor with a positive result is one more than the divisor digits.
NegExp: XDp placed in left most position. Dexp number of leading zeros right shifted into MSD, losing rightmost digits for each shift.

# Invert XDP




# Operation Chaining (Post processing the result of an operation to support chaining)



9+0.0000001=9.0000001
99999999+.1=99999999
99+0.0000001=99

# Addition Overflow and Underflow

DPFirst=1

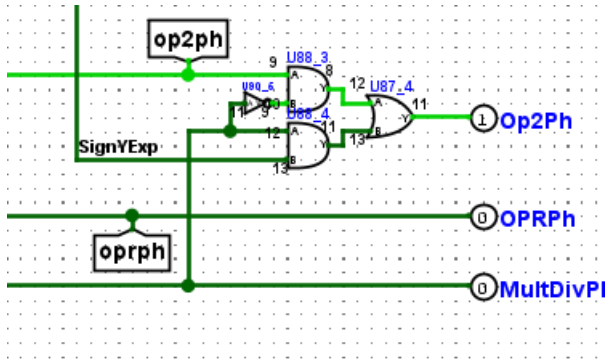
Dintovrfl=1

| | | | OP2(X) | | 10000000 | 1000000 | 100000 | 10000 | 1000 | 100 | 10 | 1 | | 0.1 | 0.01 | 0.001 | 0.0001 | 0.00001 | 0.000001 | 0.0000001 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | Xexp(+), Yexp(+) | | | | | | | | | Xexp(-),Yexp(+) | | | | | | |
| 0000 | 0 | | OP1 (Y) | exp | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | | -1 | -2 | -3 | -4 | -5 | -6 | -7 |
| 0001 | 1 | -15 | 10000000 | 7 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
| 0010 | 2 | -14 | 1000000 | 6 | -1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
| 0011 | 3 | -13 | 100000 | 5 | -2 | -1 | 0 | 1 | 2 | 3 | 4 | 5 | | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
| 0100 | 4 | -12 | 10000 | 4 | -3 | -2 | -1 | 0 | 1 | 2 | 3 | 4 | | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
| 0101 | 5 | -11 | 1000 | 3 | -4 | -3 | -2 | -1 | 0 | 1 | 2 | 3 | | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 0110 | 6 | -10 | 100 | 2 | -5 | -4 | -3 | -2 | -1 | 0 | 1 | 2 | | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 0111 | 7 | -9 | 10 | 1 | -6 | -5 | -4 | -3 | -2 | -1 | 0 | 1 | | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 1000 | 8 | -8 | 1 | 0 | -7 | -6 | -5 | -4 | -3 | -2 | -1 | 0 | | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 1001 | 9 | -7 | | | | | | | | | | | | | | | | | | |
| 1010 | 10 | -6 | 0.1 | -1 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 | | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| 1011 | 11 | -5 | 0.01 | -2 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | | -1 | 0 | 1 | 2 | 3 | 4 | 5 |
| 1100 | 12 | -4 | 0.001 | -3 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | | -2 | -1 | 0 | 1 | 2 | 3 | 4 |
| 1101 | 13 | -3 | 0.0001 | -4 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | -4 | | -3 | -2 | -1 | 0 | 1 | 2 | 3 |
| 1110 | 14 | -2 | 0.00001 | -5 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | -5 | | -4 | -3 | -2 | -1 | 0 | 1 | 2 |
| 1111 | 15 | -1 | 0.000001 | -6 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | -6 | | -5 | -4 | -3 | -2 | -1 | 0 | 1 |
| | | | 0.0000001 | -7 | -14 | -13 | -12 | -11 | -10 | -9 | -8 | -7 | | -6 | -5 | -4 | -3 | -2 | -1 | 0 |
| | | | | | Xexp(+), Yexp(-) | | | | | | | | | Xexp(-), Yexp(-) | | | | | | |

**Dexp Counter values for All Op1 and Op2 Combinations**

Add Overflow (DintOvrfl=1)(SignYexp=0)(SignXexp=0)
OR
(DintOvrfl=1)(SignYexp=1)(SignXexp=1)

Overflow happens when Op1 and Op2 are both 8-digit integers and the sum results in a final carry out digit 8.




Op2(X) is DPFirst AND DintOVrfl

Add Underflow (DintOvrfl=1)(SignYexp=0)(SignXexp=1)
Use Yreg

Add Underflow (DintOvrfl=1)(SignYexp=1)(SignXexp=0)
Use Xreg

Op2(X) is NOT DintOvrfl AND NOT DPFirst

**Because SignYexp is only available to Mult and Div (due to arch oversight) I can't Use these expressions to detect underflow.**

# Multiplication and Division Overflows and Underflows

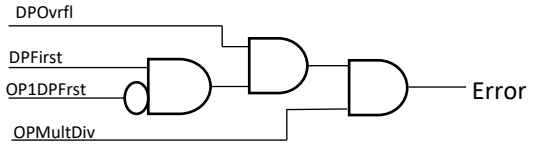
OP2

| A div C… | | 0.0000001 | 0.0000001 | 0.000001 | 0.000001 | 0.00001 | 0.00001 | 0.0001 | 0.0001 | 0.001 | 0.001 | 0.01 | 0.01 | 0.1 | 0.1 | 0 | 0 | 1 | 1 | 10 | 10 | 100 | 100 | 1000 | 1000 | 10000 | 10000 | 100000 | 100000 | 1000000 | 1000000 | 10000000 | 10000000 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | -7 | -7 | -6 | -6 | -5 | -5 | -4 | -4 | -3 | -3 | -2 | -2 | -1 | -1 | | | 0 | 0 | 1 | 1 | 2 | 2 | 3 | 3 | 4 | 4 | 5 | 5 | 6 | 6 | 7 | 7 |
| | | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | | | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp | Dexp | Sexp |
| 0.0000001 | -7 | 0 | -14 | -1 | -13 | -2 | -12 | -3 | -11 | -4 | -10 | -5 | -9 | -6 | -8 | | | -7 | -7 | -8 | -6 | -9 | -5 | -10 | -4 | -11 | -3 | -12 | -2 | -13 | -1 | -14 | 0 |
| 0.000001 | -6 | 1 | -13 | 0 | -12 | -1 | -11 | -2 | -10 | -3 | -9 | -4 | -8 | -5 | -7 | | | -6 | -6 | -7 | -5 | -8 | -4 | -9 | -3 | -10 | -2 | -11 | -1 | -12 | 0 | -13 | 1 |
| 0.00001 | -5 | 2 | -12 | 1 | -11 | 0 | -10 | -1 | -9 | -2 | -8 | -3 | -7 | -4 | -6 | | | -5 | -5 | -6 | -4 | -7 | -3 | -8 | -2 | -9 | -1 | -10 | 0 | -11 | 1 | -12 | 2 |
| 0.0001 | -4 | 3 | -11 | 2 | -10 | 1 | -9 | 0 | -8 | -1 | -7 | -2 | -6 | -3 | -5 | | | -4 | -4 | -5 | -3 | -6 | -2 | -7 | -1 | -8 | 0 | -9 | 1 | -10 | 2 | -11 | 3 |
| 0.001 | -3 | 4 | -10 | 3 | -9 | 2 | -8 | 1 | -7 | 0 | -6 | -1 | -5 | -2 | -4 | | | -3 | -3 | -4 | -2 | -5 | -1 | -6 | 0 | -7 | 1 | -8 | 2 | -9 | 3 | -10 | 4 |
| 0.01 | -2 | 5 | -9 | 4 | -8 | 3 | -7 | 2 | -6 | 1 | -5 | 0 | -4 | -1 | -3 | | | -2 | -2 | -3 | -1 | -4 | 0 | -5 | 1 | -6 | 2 | -7 | 3 | -8 | 4 | -9 | 5 |
| 0.1 | -1 | 6 | -8 | 5 | -7 | 4 | -6 | 3 | -5 | 2 | -4 | 1 | -3 | 0 | -2 | | | -1 | -1 | -2 | 0 | -3 | 1 | -4 | 2 | -5 | 3 | -6 | 4 | -7 | 5 | -8 | 6 |
| OP1 | 0 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 1 | 0 | 7 | -7 | 6 | -6 | 5 | -5 | 4 | -4 | 3 | -3 | 2 | -2 | 1 | -1 | | | 0 | 0 | -1 | 1 | -2 | 2 | -3 | 3 | -4 | 4 | -5 | 5 | -6 | 6 | -7 | 7 |
| 10 | 1 | 8 | -6 | 7 | -5 | 6 | -4 | 5 | -3 | 4 | -2 | 3 | -1 | 2 | 0 | | | 1 | 1 | 0 | 2 | -1 | 3 | -2 | 4 | -3 | 5 | -4 | 6 | -5 | 7 | -6 | 8 |
| 100 | 2 | 9 | -5 | 8 | -4 | 7 | -3 | 6 | -2 | 5 | -1 | 4 | 0 | 3 | 1 | | | 2 | 2 | 1 | 3 | 0 | 4 | -1 | 5 | -2 | 6 | -3 | 7 | -4 | 8 | -5 | 9 |
| 1000 | 3 | 10 | -4 | 9 | -3 | 8 | -2 | 7 | -1 | 6 | 0 | 5 | 1 | 4 | 2 | | | 3 | 3 | 2 | 4 | 1 | 5 | 0 | 6 | -1 | 7 | -2 | 8 | -3 | 9 | -4 | 10 |
| 10000 | 4 | 11 | -3 | 10 | -2 | 9 | -1 | 8 | 0 | 7 | 1 | 6 | 2 | 5 | 3 | | | 4 | 4 | 3 | 5 | 2 | 6 | 1 | 7 | 0 | 8 | -1 | 9 | -2 | 10 | -3 | 11 |
| 100000 | 5 | 12 | -2 | 11 | -1 | 10 | 0 | 9 | 1 | 8 | 2 | 7 | 3 | 6 | 4 | | | 5 | 5 | 4 | 6 | 3 | 7 | 2 | 8 | 1 | 9 | 0 | 10 | -1 | 11 | -2 | 12 |
| 1000000 | 6 | 13 | -1 | 12 | 0 | 11 | 1 | 10 | 2 | 9 | 3 | 8 | 4 | 7 | 5 | | | 6 | 6 | 5 | 7 | 4 | 8 | 3 | 9 | 2 | 10 | 1 | 11 | 0 | 12 | -1 | 13 |
| 10000000 | 7 | 14 | 0 | 13 | 1 | 12 | 2 | 11 | 3 | 10 | 4 | 9 | 5 | 8 | 6 | | | 7 | 7 | 6 | 8 | 5 | 9 | 4 | 10 | 3 | 11 | 2 | 12 | 1 | 13 | 0 | 14 |

**Dexp and Sexp Counter Values From All Combinations of Op1 and Op2**

Division Overflows
Anytime Dexp is positive number greater than 7
Within the positive exp1, negative exp2 quadrant

(DPOvrfl=1)(SignYexp=0)(SignXexp=1)

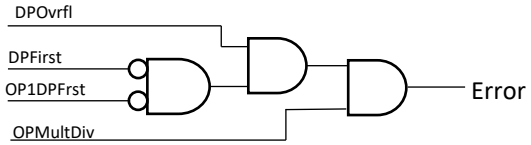



Div Underflows

(DPOvrfl=1)(SignYexp=1)(SignXexp=0)

Mult Overflows
Anytime Sexp is positive number greater than 7
Within the positive exp1, positive exp2 quadrant

(DPOvrfl=1)(SignYexp=SignXexp=0)




Mult Underflows

(DPOvrfl=1)(SignYexp=SignXexp=1)